



الجامعة السورية الخاصة
SYRIAN PRIVATE UNIVERSITY

الجمهورية العربية السورية
الجامعة السورية الخاصة
كلية هندسة المعلوماتية

نظام إدارة مركز صحي

اعداد:

محمد براء العمار

اشراف :

د.شادي بليدي

آب/2026

Syrian Arab Republic
Syrian Private University
Faculty of Information
Engineering



“Medical Center Management System”

Prepared by:

Mohammad Baraa Al-Ammar

Supervised by:

Dr. Shadi Blidi

August/2026

Supervisor Certification

.....
prepared by

..... was made
under my supervision at Department of Computer and Information Engineering –
Faculty of Computer & Communication Engineering in partial fulfillment of the
Requirements for the Degree of Bachelors of Software and information System
Engineering.

Academic Rank:..... Name:..... Signature:

Date:

الملخص

هذا النظام المتكامل لإدارة المركز الطبي يعمل على أتمتة وتنظيم العمليات الإدارية والطبية والصيدلانية في منصة واحدة. يدير النظام المرضى والمواعيد والطاقم الطبي ومخزون الصيدلانية والتواصل بين المريض والطبيب والأنشطة اليومية.

يمكن لموظفي الاستقبال تسجيل المرضى، وحجز المواعيد مع التحقق من توفر الأطباء في الوقت الفعلي، وإدارة سجلات الزيارات. يمكن للأطباء عرض جداولهم، وإكمال المواعيد السابقة، وإضافة الملاحظات الطبية مع التشخيصات والوصفات الطبية، وإدارة مواعيد توفرهم الأسبوعية. يمكن للمرضى التسجيل، وحجز المواعيد أو إعادة جدولتها أو إلغاؤها، وعرض سجلهم الطبي، وتقييم الأطباء أو تقديم شكاوى، والردشة المباشرة مع أطباءهم مع إمكانية إرفاق الملفات.

يدير قسم الصيدلانية فئات الأدوية، وكميات المخزون، وتنبيهات انخفاض المخزون، وتواريخ انتهاء الصلاحية، والأسعار. يمكن للصيدلة صرف الأدوية بناءً على الوصفات الطبية، وإصدار الفواتير، وتسجيل تعديلات المخزون، وعرض تقارير المبيعات اليومية والشهرية.

تشمل الميزات الإضافية نظام إشعارات طارئ لإرسال تنبيهات فورية للأطباء، وإشعارات تأكيد المواعيد ومكافآت الولاء، وروبوت محادثة ذكي للأسئلة الشائعة وساعات العمل والموقع واختصاصات الأطباء وتعليمات الحجز والنماذج القابلة للتحميل. يتتبع نظام الولاء الزيارات المكتملة تلقائياً ويمنح مكافآت كل أربع زيارات.

يضمن التحكم في الوصول القائم على الأدوار أمن البيانات للمسؤولين والأطباء وموظفي الاستقبال والصيدلة والمرضى. يدير المسؤولون حسابات الطاقم ويراجعون جميع التقييمات والشكاوى.

يحسّن هذا النظام كفاءة سير العمل، ويرفع جودة الخدمات الصحية، ويقلل الأخطاء الإدارية والطبية، ويخلق بيئة عمل أكثر تنظيماً ودقة وفعالية من حيث الوقت.

ABSTRACT

This medical center management system automates and organizes administrative, medical, and pharmaceutical operations in a single platform. It supports managing patients, appointments, medical staff, pharmacy inventory, patient doctor communication, and daily activities.

Receptionists can register patients, book appointments with real-time doctor availability checks, and manage visit records. Doctors can view schedules, complete past appointments, add medical notes with diagnoses and prescriptions, and manage their weekly availability. Patients can register, book, reschedule, or cancel appointments, view medical history, rate doctors or submit complaints, and chat directly with their doctors including file attachments.

The pharmacy module manages medicine categories, stock quantities, low-stock alerts, expiry dates, and pricing. Pharmacists can dispense medicines against prescriptions, generate invoices, log stock adjustments, and view daily and monthly sales reports.

Additional features include an emergency notification system for real time alerts to doctors, appointment confirmations and loyalty reward notifications, and an intelligent chat bot, working hours, location, doctor specialties, booking instructions, and downloadable forms. A loyalty system automatically tracks completed visits and grants rewards on every fourth visit.

Role based access control ensures data security for administrators, doctors, receptionists, pharmacists, and patients. Administrators manage staff accounts and review all ratings and complaints.

This system improves workflow efficiency, enhances healthcare service quality, reduces administrative and medical errors, and creates a more organized and time-efficient working environment

Contents

Chapter One	1
Introduction and Definition of the Project	1
1.1 Bringing The Problem	2
1.2 Project Concept.....	2
1.3 Aim of the Project	2
1.4 Documentation Description.....	3
1.4.1 Project Study	3
1.4.2 Theoretical Study of the Project	3
1.5 Analytical and Design Study of the Project	3
1.6 Conclusion and Recommendations	3
1.7 System Diagrams.....	4
Chapter Two	6
Project topic and reference study	6
2.1 Introduction	7
2.2 The Basic Concepts	7
2.3 Reference Study	8
2.4 Importance of Services	9
2.5 Types of Users.....	9
2.6 Similar Applications:	10
2.7 Comparison of Similar Applications.....	12
Chapter Three Analytical Study of the Proposed System	13
3. Analytical Study of the Proposed System	14
3.1 Requirements Document:.....	14
3.1.1 Documentation and Auxiliary Reports:	14
3.2 Feasibility:	15
3.3 SRS Document:	17
3.3.4 Performance Requirements:.....	20

3.3.5 Design Requirements:	21
3.3.6 Project Limitations:	22
3.4Gantt Chart:	24
3.5 Analysis chart of project	25
3.5.1 Use-Case Diagram (Low Level):	25
3.5.2 Use-Case Diagram (High Level):	26
.....	26
3.5.3 Activity diagram (add category)	27
3.5.4 Activity Diagram (edit medicine diagram)	28
.....	28
3.5.5 Activity diagram (dispense diagram)	29
3.5.6 Activity diagram (view available medicines)	30
3.5.7 Activity diagram (view daily sales)	31
3.5.8 Activity diagram (assign category)	32
3.5.9 Activity diagram (display faqs)	33
3.5.10 Activity Diagram (view complaints)	34
3.5.11 Activity diagram (rate a doctor)	35
3.5.12 Activity diagram (add specialty)	36
3.5.13 Sequence diagram (log in)	37
3.5.14 Sequence diagram (Add catagory)	38
3.5.15 Sequence Diagram (Add Pharmacist)	39
3.5.16 Sequence Diagram (Edit medicine Info)	40
3.5.17 Sequence Diagram (Despines Medicine)	41
3.5.18 Sequence Diagram (Notify Low medicine)	42
3.5.19 Sequence Diagram(View Available Medicine)	42
3.5.20 Sequence Diagram (View Daily Sale Record)	43
3.5.21 Sequence Diagram (Assign Category)	43
3.5.22 Sequence Diagram(Display FAQs and Answer)	44
3.5.23 Sequence Diagram (Explain Appointment Booking Process)	45

3.5.24	Sequence Diagram (Patient Chat with Doctor).....	46
3.5.25	Sequence Diagram (Send Emergency Notification).....	47
3.5.26	Sequence Diagram (Patient Rating a Doctor)	48
3.5.27	Sequence Diagram (Register a Patient)	49
3.5.28	Sequence Diagram (Doctor Managing Weekly Availability).....	50
3.5.1	Use case Specification:	51
3.5.25.1	Login.....	51
Table 1	log in	51
3.5.25.2	Add Doctor.....	52
Table 2	add doctor.....	52
3.5.25.3	Add Receptionist	53
Table 3	add receptionist	53
3.5.25.4	Add New Medicine Category	54
Table 4	add new medicine category.....	54
3.5.25.5	Edit Medicine Information.....	55
Table 5	edit medicine information	55
3.5.25.6	Delete Medicine Category	56
Table 6	delete medicine category.....	56
3.5.25.7	View Available Medicines	57
Table 7	view available medicines	57
3.5.25.8	Search Medicine by Name or Code	58
Table 8	search by name or code.....	58
3.5.25.9	Receive Prescription from Doctor	59
Table 9	receive prescription from doctor	59
3.5.25.10	Dispense Medicine.....	60
Table 10	dispense medicine	60
3.5.25.11	Notify Low Medicine Stock	61
Table 11	notify low medicine stock	61
3.5.25.12	Record Medicine Expiration Dates	62

Table 12 record medicine expiration dates	62
3.5.25.13 View Daily Sales Record	63
Table 13 view daily sales record	63
3.6 Entity Relationship Diagram (ERD) Low Level :	64
3.7 Entity Relationship Diagram (ERD) High Level :	65
3.7 Database Tables	66
3.8 Test Cases:	74
3.9 Requirements Tracking Matrix:	80
3.10 System Architecture:	85
3.10.1 Client-Server Architecture:	85
3.10.2 Model-View-Controller (MVC) Architecture	85
3.10.3 Features of this Architecture:	86
3.10.4 Why Use this Architecture for this System?	86
3.10.6 Client_Server Diagram /MVC Diagram :	87
4.1 Class Diagram(Low Level):	90
4.3 Design Models at the Physical Level:	92
4.4 System Interface Diagram	94
Chapter Five	95
Implementation of Project	95
5.1 Programming Resources:	96
5.1.1 Laravel Framework (Backend Framework):	96
5.1.2 Tailwind CSS (Frontend Framework):	96
5.1.3 Alpine.js (Frontend Interactivity):	97
5.2.1 Laravel Artisan (Command Line Tool):	97
5.3 Technologies Used:	97
5.3.1 HTML5:	98
5.3.2 CSS3:	98
5.3.3 JavaScript:	98
5.3.4 PHP (Hypertext Preprocessor):	98

5.4 Interface Pages:	99
5.4.1 (All Users) Login Page:	99
5.4.2 (Admin) Dashboard:	99
5.4.3 (Admin) Manage Staff:	99
5.4.3.2 (Admin) Edit Staff:	99
5.4.3.3 (Admin) Delete Staff:	99
5.4.4 (Admin) View Ratings:	99
5.4.5 (Admin) View Complaints:	100
5.4.7 (Receptionist) Dashboard	100
5.4.8 (Receptionist) Register Patient:	100
5.4.9 (Receptionist) Book Appointment	100
5.4.10 (Receptionist) Manage Appointments:	100
5.4.11 (Receptionist) Send Emergency Notification:	100
5.4.12 (Receptionist) View Monthly Reports:	100
5.4.13 (Doctor) Dashboard:	101
5.4.14 (Doctor) View Appointments	101
5.4.15 (Doctor) View Patient Medical History:	101
5.4.16 (Doctor) Add Medical Record:	101
5.4.17 (Doctor) Manage Availability:	101
5.4.18 (Doctor) Chat with Patients:	101
5.4.19 (Doctor) Profile:	101
5.4.20 (Pharmacist) Dashboard:	101
5.4.21 (Pharmacist) Manage Categories:	102
5.4.22 (Pharmacist) Manage Medicines:	102
5.4.23 (Pharmacist) View Prescriptions:	102
5.4.24 (Pharmacist) Dispense Prescription:	102
5.4.25 (Pharmacist) View Invoices:	102
5.4.27 (Pharmacist) View Sales Reports:	102
5.4.28 (Pharmacist) Profile:	102

5.4.29 (Patient) Dashboard:	103
5.4.30 (Patient) Book Appointment:	103
5.4.31 (Patient) Manage Appointments:.....	103
5.4.32 (Patient) View Medical History:	103
5.4.33 (Patient) Chat with Doctor:.....	103
5.4.34 (Patient) Rate Doctor:.....	103
5.4.35 (Patient) Submit Complaint:.....	103
5.4.36 (Patient) Chatbot Assistant	103
5.4.37 (Patient) Notifications:.....	103
5.4.38 (Patient) Profile:	104
5.5 Interfaces Screen:.....	104
Chapter Six.....	106
Test and Result	106
6.1 Introduction:	107
6.1.1 Tests During Development:	107
6.1.1.1 Software Unit Testing:	107
6.1.1.2 Integration Test:	107
6.1.1.3 Quality Tests:	107
6.1.2 Tests After Development:	108
6.1.2.1 Google Lighthouse:	108
Chapter Seven	109
Consultation and Future Prospects	109
7. Conclusion and Future Prospects	110
7.1 Conclusion:	110
7.2 Future Prospects:	110
Chapter Eight	111
Project Management.....	111
8.1 Project Description:	112
8.2 Branch and Merge Strategy	113

8.3 Team Members and Responsibilities	114
8.4 Development Workflow	114
8.5 Tags and Version Management	115
Chapter Nine	116
References	116

Figure 1 chapter 2 comparison table	12
Figure 2 chapter 3.1 ganttchart.....	24
Figure 3 chapter 3.2 use case low level	25
Figure 4 chapter3.3. use case high level	26
Figure 5 chapter3.5 ad add category	27
Figure 6 chapter3.6 ad edit medicine diagram	28
Figure 7 chapter3.7 ad dispense diagram	29
Figure 8 chapter3. 8 ad view available medicines	30
Figure 9 chapter3.9 ad view daily sales	31
Figure 10 chapter3.10 ad assign category	32
Figure 11 chapter3.11 ad display faqss	33
Figure 12 chapter3.12 ad view complaints	34
Figure 13 chapter3.13 ad rate a doctor	35
Figure 14 chapter3.14 ad add specialty	36
Figure 15 chapter3.15 SQ login	37
Figure 16 chapter3.16 sq add category	38
Figure 17 chapter3 3.17 sq add staff.....	39
Figure 18 chapter3.18 sq edit medicine info	40
Figure 19 chapter3.19 sq dispince medicine	41
Figure 20 chapter3.20 sq notify low medicine	42
Figure 21 chapter3.21 sq View Available Medicine	42
Figure 22 chapter3.22 sq View Daily Sale Record	43
Figure 23 chapter3.23 sq Assign Category	43
Figure 24 chapter3.24 sq Display FAQs and Answer	44
Figure 25 chapter3.25 sq Explain Appointment Booking Process	45
Figure 26 chapter3.26 sq Patient Chat with Doctor	46
Figure 27 chapter3.27 sq Send Emergency Notification	47
Figure 28 chapter3.28 sq Patient Rating a Doctor	48
Figure 29 chapter3.29 sq Register a Patient.....	49

Figure 30 chapter3.30 sq Doctor Managing Weekly Availability	50
Figure 31 chapter3.31 ERD low level	64
Figure 32 chapter3.32 ERD high level	65
Figure 33 chapter3 RTM.....	84
Figure 34 chapter3.34 client server	87
Figure 35 chapter3.35 mvc.....	88
Figure 36 chapter3.36 mvc/clinet server	88
Figure 37 chapter4.1class diagram low level	90
Figure 38 chapter4.2 class diagram high level	91
Figure 39 chapter4.3 System Interface Diagram.....	94
Figure 40 chapter4.4 html.....	98
Figure 41 chapter4.5 php	98
Figure 42 chapter4.6 MySQL.....	98
Figure 43 chapter4.7 Admin Dashboard	104
Figure 44 chapter4.8 Reception Dashboard	104
Figure 45 chapter6.1 Google test	108
Figure 46 chapter8.1 githut repository	112
Figure47 chapter8.2 bracnhing.....	113
Figure 48 chapter8.3 tags	115

Table 1 chapter 3.1 log in.....	51
Table 2 chapter 3.2 add doctor.....	52
Table 3 chapter 3.3 add receptionist.....	53
Table 4 chapter 3.4 add new medicine category	54
Table 5 chapter 3.5 edit medicine information	55
Table 6 chapter 3.6 delete medicine category	56
Table 7 chapter 3.7 view available medicines	57
Table 8 chapter 3.8 search by name or code	58
Table 9 chapter 3.9 receive prescription from doctor	59
Table 10 chapter 3.10 dispense medicine	60
Table 11 chapter 3.11 notify low medicine stock	61
Table 12 chapter 3.12 record medicine expiration dates	62
Table 13 chapter 3.13 view daily sales record.....	63
Table 14 chapter 3.14 Structure for Admin.....	66
Table 15 chapter 3.15 Doctor Table.....	67

Table 16 chapter 3.16 Receptionist Table	68
Table 17 chapter 3.17 Pharmacist Table	69
Table 18 chapter 3.18 Patient Table	70
Table 19 chapter 3.19 Appointment Table	71
Table 20 chapter 3.20 Medical Record Table	72
Table 21 chapter 3.21 Prescription Table	73
Table 22 chapter 3.22 Medicine Category Table	73

Chapter One

Introduction and Definition of the Project

1.1 Bringing The Problem

Medical centers face challenges managing patient records, appointments, staff coordination, and pharmacy inventory, often using paper-based or disconnected systems. This causes slow processes, data access issues, and frequent human errors including appointment conflicts, lost or duplicate records, poor medication stock management, and weak patient-doctor communication. Expired or low-stock medications go unnoticed, prescriptions lack centralized tracking, and there's no automated system for emergencies or loyalty programs, all of which harm healthcare quality and patient satisfaction.

1.2 Project Concept

This Medical Center Management System automates and integrates all administrative, medical, and pharmaceutical operations into a single digital platform, replacing manual processes with a unified database for patient records, appointments, medical notes, doctor availability, pharmacy management, communication, and feedback.

Built on a role-based architecture, it serves five user types administrators, receptionists, doctors, pharmacists, and patients each with tailored permissions and interfaces.

1.3 Aim of the Project

The project provides a complete integrated platform for medical centers, enabling all staff to perform their duties efficiently without time or location constraints.

The system centralizes patient data (personal info, medical history, allergies, visits) in one secure database, automates appointments with real-time doctor availability checks, and lets doctors manage schedules, view histories, and add diagnoses and prescriptions. It includes a full pharmacy management system for tracking stock, categories, expiry dates, alerts, and pricing, with prescriptions linked directly to invoices. Patients can communicate with doctors via an

integrated chat with file sharing, earn loyalty rewards automatically every four visits, and get answers from a chatbot about FAQs, hours, location, specialties, and booking. Administrators can manage staff accounts and review all ratings and complaints.

The system emphasizes ease of use, data accuracy, security, and flexibility with tailored interfaces for all roles.

1.4 Documentation Description

This document provides a detailed overview of the stages of project completion, including studying the project, defining its objectives, analyzing requirements, designing the system architecture, and implementing the solution. The project was studied theoretically and then applied practically to ensure it meets the real-world needs of medical centers.

1.4.1 Project Study

This phase defined project objectives, identified target users, analyzed core problems faced by medical centers, and determined the most effective approach to develop a comprehensive digital solution addressing all identified issues.

1.4.2 Theoretical Study of the Project

The project was studied theoretically and then applied practically.

1.5 Analytical and Design Study of the Project

Includes a detailed description of project components and how they are analyzed, designed, investigated, and tested.

1.6 Conclusion and Recommendations

Presents lessons learned and suggestions for modification and development.

1.7 System Diagrams

The system diagram for the Medical Center Management System is a comprehensive design that includes all components required for effective development and operation. It includes a set of drawings and diagrams that accurately describe each part of the system, from the database to the user interface.

The system diagram includes the following components:

1. Database: Contains all information about patients, doctors, appointments, medical records, pharmacy inventory, and prescriptions.
2. User Interface: Provides tailored interfaces for administrators, receptionists, doctors, pharmacists, and patients to perform their tasks efficiently.
3. Appointment Management System: Manages booking, rescheduling, cancellation, and real-time doctor availability checking.
4. Pharmacy Management System: Tracks medicine stock, categories, expiry dates, low-stock alerts, pricing, and invoice generation.
5. Communication System: Enables direct chat between patients and doctors with file attachment support.
6. Notification System: Sends emergency alerts to doctors, appointment confirmations, and loyalty reward notifications.
7. Chat bot System: Answers patient inquiries about FAQs, working hours, location, doctor specialties, and booking instructions.

8. Loyalty System: Automatically tracks completed visits and grants rewards every four visits.
9. Security and Access Control System: Ensures data protection through role-based access for all user types.
10. Reporting System: Generates sales reports, visit statistics, and performance summaries.

Chapter Two

Project topic and reference study

2.1 Introduction

In this chapter, we will study the topic of the project and the reference study for this medical center management system.

2.2 The Basic Concepts

Patient Management

The process of registering and organizing patient data within the system, including personal information, medical history, previous visits, and health conditions. This ensures easy access to patient data and improves the accuracy of medical documentation.

Medical Appointment Management

Scheduling and organizing examination and consultation appointments while defining doctors' working hours and preventing scheduling conflicts. The system also sends automatic notifications to remind patients and doctors of upcoming appointments.

Medical Staff Management

Organizing the data of doctors and medical staff, defining their specialties, working hours, and system permissions. This improves coordination between medical and administrative teams.

Medical Records Management

Storing and archiving medical records electronically, enabling easy, secure, and organized access to patient information. This protects medical data and ensures quick retrieval when needed.

Pharmacy and Inventory Management

Managing medicine categories, tracking stock quantities, setting low-stock thresholds, monitoring expiry dates, and managing pricing. It also handles prescription processing, invoice generation for dispensed medicines, and logging damaged or returned stock adjustments.

Patient-Doctor Communication

Direct messaging between patients and their doctors through an integrated chat system supporting text messages and file attachments, with a complete conversation history maintained for each patient-doctor pair.

Ratings and Complaints

Patients can submit ratings and complaints about their experiences with doctors, which administrators can review and manage separately.

Loyalty Rewards System

Automatically tracks completed patient visits and grants rewards on every fourth completed visit, sending automatic notifications to encourage patient engagement.

Medical and Administrative Reports

Generates monthly appointment statistics for receptionists and daily/monthly sales reports for pharmacists, helping management monitor operations and make data-driven decisions.

2.3 Reference Study

A medical center management system is a digital platform that manages and organizes all medical and administrative operations electronically, using modern technology and fast communications. This system aims to facilitate patient registration, appointment scheduling, medical record keeping, and pharmacy management, improving service quality and reducing costs and time spent on manual processes. It also provides innovative solutions to challenges facing medical centers, such as administrative errors, appointment conflicts, poor inventory management, and weak patient-doctor communication.

A medical center management system differs from traditional paper-based systems in many aspects, including:

1. Management: The system is managed electronically, while traditional systems rely on manual paper-based processes.
2. Speed: Appointments, prescriptions, and reports are processed faster due to the use of modern technology.

3. Cost: Traditional systems require higher costs for paper, printing, filing, and storage, while costs are significantly reduced in digital systems.
4. Access: Patients and staff can easily access data and records from anywhere, while traditional systems require physical presence and manual file retrieval.
5. Security: The level of security is improved through encryption, user authentication, and role-based access control, while paper records can be lost, damaged, or accessed by unauthorized individuals.

2.4 Importance of Services

1. Saving time and effort: The system facilitates the management of patients, appointments, medical records, and pharmacy operations faster and more effectively, saving time and effort for medical staff and patients alike.
2. Reducing costs: Costs are reduced by eliminating paper-based records, minimizing human errors, and automating repetitive administrative tasks.
3. Increased security: The system helps increase security through role-based access control, ensuring that each user can only access data relevant to their role, protecting patient privacy and sensitive medical information.
4. Improving service quality: Patients can book appointments online, communicate directly with their doctors, receive appointment notifications, view their medical history, and benefit from a loyalty rewards program — all improving the overall patient experience.
5. Pharmacy efficiency: Automated stock tracking, low-stock alerts, expiry date monitoring, and prescription-based dispensation reduce medication errors and improve inventory management.
6. Environmental Sustainability: The system helps reduce the use of paper and materials needed for traditional medical record keeping, contributing to environmental preservation.

2.5 Types of Users

1. Administrators: Manage staff accounts, review all ratings and complaints submitted by patients, and oversee overall system operations.
2. Receptionists: Register patients, book appointments with real-time doctor availability checks, and manage visit records.
3. Doctors: View their schedules, complete past appointments, add medical notes with diagnoses and prescriptions, and manage their weekly availability.
4. Pharmacists: Manage medicine inventory, dispense medicines against prescriptions, generate invoices, log stock adjustments, and view sales reports.
5. Patients: Register accounts, book, reschedule, or cancel appointments, view medical history, rate doctors or submit complaints, and communicate directly with their doctors.

2.6 Similar Applications:

2.6.1 Practo: is a global electronic platform aimed at facilitating the management of clinics and medical centers. It provides services for online appointment booking, patient data management, and doctor schedule organization. The platform helps patients easily access medical services, while also supporting doctors in efficiently managing their clinics.

2.6.2 Zocdoc: This platform provides online medical appointment booking services, allowing patients to search for doctors based on specialty and location and book appointments easily. It also helps doctors organize their schedules and improve the efficiency of communication with patients.

Key features include instant booking, verified patient reviews, insurance acceptance filtering, and appointment reminders. However, Zocdoc focuses primarily on appointment booking and does not include pharmacy management, in-clinic medical records, or direct patient-doctor chat features.

2.6.3 Clinic Management Systems (Open-source): Several open-source clinic management systems exist, such as OpenEMR and HospitalRun. These provide basic features for patient records, appointments, and billing.

2.7 Comparison of Similar Applications

Feature	Angel Infotech CMS	Hellog CMS	Ray by Practo	Our system
Add new patient	✓	✓	✓	✓
Edit patient information	✓	✗	✗	✓
Search patient	✗	✓	✗	✓
Book appointment	✓	✓	✓	✓
Edit appointment	✓	✓	✗	✓
Cancel appointment	✓	✗	✓	✓
View doctor's daily schedule	✓	✗	✓	✓
Manage doctor availability	✗	✓	✓	✓
Add medical notes for patient	✗	✓	✗	✓
View monthly reports	✓	✗	✓	✓

Figure 1 chapter 2 comparison table

Chapter Three Analytical Study of the Proposed System

3. Analytical Study of the Proposed System

This chapter presents all aspects of the project in a practical way, where the functional and non-functional requirements of the project will be detailed, the mechanism for achieving them and the cases of their use, and then testing their validity and achieving the desired goals. The analysis stage is the basis of the project development stages — any error in this stage leads to an error in all the following stages and can lead to the failure of the entire project, or at the very least the failure of one of the project's goals, defining concepts, objectives and the framework of the problem, deleting illogical concepts and understanding the project more.

3.1 Requirements Document:

This chapter presents all aspects of the project in a practical way, as it will detail the functional and non-functional requirements of the project, the mechanism for achieving them and their use cases, then testing their validity and achieving the desired goals.

Given the analysis stage is the basis of the project development stages, any error in this stage leads to an error in all the following stages and can lead to the failure of the entire project, or at least the failure of one of the project's goals.

In this project, information was collected in order to conduct an analytical study of the system in order to obtain documented and correct information, which leads to the success of the project.

3.1.1 Documentation and Auxiliary Reports:

Some references on the Internet were relied upon, in addition to benefiting from some previous reports and projects, as their importance was to give a broader and more comprehensive perception of the idea, while giving perceptions of the non-functional services that the system should provide.

3.2 Feasibility:

The economic feasibility of implementing a medical center management system project is the process of evaluating and analyzing the financial and economic aspects of the project. Economic feasibility aims to determine whether the project brings expected financial benefits and is worth investing in. The economic viability assessment includes several important factors, including:

1. **Implementation cost:** The total cost of developing and operating the project must be determined, including the cost of developing the platform, the cost of human resources, the cost of marketing, and any other related costs.
2. **Projected revenue:** Projected revenue should be estimated, such as subscription fees or monthly fees for medical centers, and revenue from additional services or support contracts.
3. **Maintenance and operation cost:** The cost of maintaining and operating the project must be taken into account, including hosting and storage costs, updates and upgrades, and technical support.
4. **Benefit Analysis:** The expected benefits of the project should be identified, such as reducing administrative errors, improving patient satisfaction, increasing the number of patients served per day, and saving time and effort for medical staff.
5. **Cost-Benefit Analysis:** A detailed cost-benefit analysis must be performed to assess whether expected benefits exceed expected costs and at what rate.
6. **Risk Assessment:** The potential risks associated with the implementation of the medical center management system must be analyzed and their impact on the economic and financial aspects assessed.

Using this information and financial analyses, a comprehensive economic feasibility study can be prepared to evaluate the implementation of the medical center management system project and take the appropriate decision based on the results.

The project is to create a platform for managing medical centers. Here is an example of a feasibility analysis for this project: (Please note that these figures are fictitious and are for illustrative purposes only and may vary depending on actual conditions and economic fluctuations):

1. Implementation cost:

1. Development and design of the platform: 200,000,000 SP.
2. Annual marketing costs: 15,000,000 SP.
3. The total costs of the project in the first year: 215,000,000 SP.

2. Expected Revenue:

1. Annual subscription fees for medical centers: 1,500,000 SYP per medical center.
2. The goal is to get 200 medical centers in the first year.
3. Expected revenues in the first year: 300,000,000 SP.

3. Maintenance and Operation Cost:

1. Annual hosting and operating costs: 40,000,000 SP.
2. Annual technical support and updates costs: 15,000,000 SYP.
3. Total costs of maintenance and operation in the first year: 55,000,000 SP.

4. Benefits Analysis:

1. Provide digital management for medical centers in remote areas that lack such systems.
2. Reducing administrative and medical errors and their associated costs.
3. Increasing patient satisfaction through better service and faster appointments.
4. Improving inventory management and reducing medication waste due to expired stock.

5. Cost Benefits Analysis:

The expected net return in the first year: Project revenues - (implementation + maintenance and operation costs) = 300,000,000 - (215,000,000 + 55,000,000) = 300,000,000 - 270,000,000 = 30,000,000 SP.

6. Risk Assessment:

1. Competition in the medical center management software market.
2. Challenges of expansion and increasing the number of client medical centers.
3. The impact of technology and technical dependence on project sustainability.

Based on this analysis of economic feasibility, we can make an informed decision regarding the implementation of a medical center management system project, according to the expected return, implementation cost and potential risks. These numbers can be optimized and adjusted according to the circumstances and requirements of your individual project.

3.3 SRS Document:

A document used to describe the software requirements for a specific project. The document aims to document and define the functional and non-functional requirements for the system to be developed.

An SRS document usually contains the following components:

3.3.1 Introduction:

This platform aims to provide a comprehensive and integrated experience for managing medical centers via the Internet, serving administrators, receptionists, doctors, pharmacists, and patients.

3.3.2 System Description:

1. The system is available to all authorized users at any time and from anywhere.
2. Patients can register for the first time electronically without needing to visit the medical center.
3. The system includes management of patients, appointments, medical records, pharmacy inventory, and communications.
4. The system supports role-based access control with tailored interfaces for each user type.

3.3.3 Project Requirements:

3.3.3.1 Functional Requirements:

Functional Requirements of the Administrator:

1. View dashboard with all system details and statistics.
2. Manage staff accounts (add, edit, delete doctors, receptionists, pharmacists).
3. Review all ratings and complaints submitted by patients.
4. Manage system settings and configurations.
5. Edit profile.

Functional Requirements of the Receptionist:

1. Register new patients in the system.
2. Book appointments with real-time doctor availability checks.
3. Reschedule or cancel appointments.
4. Manage visit records.
5. Send emergency notifications to doctors.
6. Edit profile.

Functional Requirements of the Doctor:

1. View personal schedule and upcoming appointments.
2. Complete past appointments and add medical notes.
3. Add diagnoses and prescriptions to patient records.
4. Manage weekly availability (set working hours and days off).
5. View patient medical history.
6. Communicate with patients via chat.
7. Edit profile.

Functional Requirements of the Pharmacist:

1. Manage medicine categories (add, edit, delete).
2. Track stock quantities and set low-stock alerts.
3. Monitor expiry dates of medicines.
4. Manage pricing information.
5. Dispense medicines against prescriptions.
6. Generate invoices for dispensed medicines.
7. Log damaged or returned stock adjustments.
8. View daily and monthly sales reports.
9. Edit profile.

Functional Requirements of the Patient:

1. Register a personal account.
2. Book, reschedule, or cancel appointments.
3. View personal medical history.
4. Rate doctors and submit complaints.
5. Communicate with doctors via chat with file attachments.

6. Receive appointment confirmations and reminders.
7. View loyalty rewards status.
8. Interact with the chatbot for FAQs and inquiries.
9. Edit profile.

3.3.3.2 Non-Functional Requirements:

1. **Usability**
The system should be easy to understand and use. Pages, buttons, and messages should be clear so that staff can learn the system quickly.
2. **Performance**
Basic actions such as logging in, searching for a patient, or opening the appointments page should respond within a few seconds under normal use.
3. **Reliability**
The system should run in a stable way during the medical center's working hours.
4. **Security**
 - Only logged-in users can access the system.
 - Each role can only see and do what is allowed for that role.
 - Passwords are stored securely in the database.
5. **Data Integrity**
All patient, appointment, and payment data should be stored in a **single database**.

3.3.4 Performance Requirements:

1. **Response Time:** Website response time should be less than 2 seconds to achieve a smooth and fast user experience. Images and content must be loaded quickly and efficiently to avoid delays.
2. **Resource Usage:** RAM usage of the host server should be moderate and not exceed the specified limit to achieve stable performance. Database queries should be optimized to reduce data retrieval time and improve read/write performance.

3. Loading Capacity: The system must withstand a high load of users during peak periods, such as morning clinic hours or vaccination campaigns. Performance must be optimized when dealing with large volumes of patient records, appointments, and pharmacy data.

4. Application Response: The application must have fast performance in carrying out operations related to appointment booking, prescription processing, and inventory updates. Search and browsing operations must be optimized to provide an interactive and smooth experience for all users.

5. Expandability: The application must be expandable to accommodate increasing numbers of patients, doctors, and medical centers. The architecture must be scalable so that additional modules and features can be easily added without affecting overall performance.

3.3.5 Design Requirements:

1. User Interface: The user interface must be simple and easy to use to enable all user types (administrators, receptionists, doctors, pharmacists, patients) to navigate and interact easily. The interface must be attractive using appropriate graphic design and colors, and must be compatible with various devices including desktop computers, tablets, and smartphones.

2. Page Layout and Structure: Pages should have a tidy and organized structure to enable users to find content and functionality easily. Each page should have a responsive design that adapts text size, images, and buttons based on screen size and device type. Pages should allow quick and flexible navigation between different sections of the platform.

3. Database Design: The database must be designed effectively to store and manage patient data, appointments, medical records, pharmacy inventory, and user information. The database must be reliable and secure to protect sensitive medical data and prevent data loss or corruption.

4. System Security: The system must be designed with high security standards to protect users' data and ensure the confidentiality of medical information. Mechanisms must be implemented to prevent unauthorized access and protect the system from electronic attacks.

6. Platform Compatibility: The platform must be compatible with popular web browsers (Chrome, Firefox, Edge, Safari)

3.3.6 Project Limitations:

1. Technology Limitations: There may be technical limitations related to programming requirements or the tools and technologies used in developing the platform. Limitations may exist regarding the scope of technical support, server hardware, and internet connection requirements.

2. Time Constraints: Time constraints related to scheduling and project execution must be considered. The set schedule for project delivery must be followed and targets must be achieved in a timely manner.

3. Resource Constraints: Limitations on financial, human, and material resources available for the project must be managed. Resources must be used effectively to achieve project objectives with the best possible utilization.

4. Security and Privacy Restrictions: Security and privacy restrictions apply to the medical center management system. Data security practices must be followed, patient privacy must be protected, and a safe and secure environment must be provided in compliance with medical data protection standards.

5. Compatibility and Responsiveness Limitations: Limitations regarding the platform's compatibility with different devices, browsers, and operating systems must be addressed. The platform must be compatible with a wide variety of devices and provide a prompt and appropriate response to the user experience.

3.4Gantt Chart:

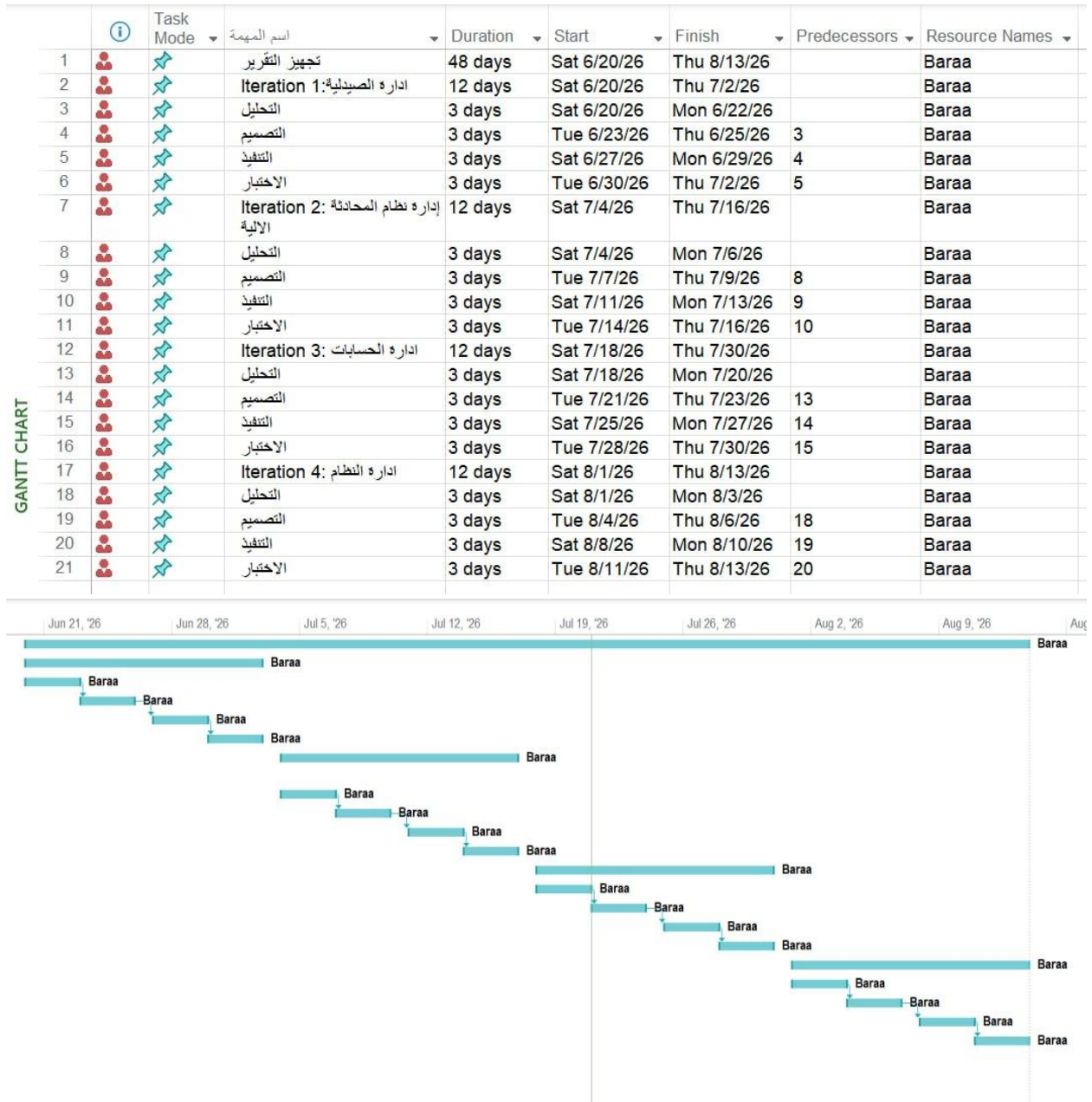


Figure 2 chapter 3.1 ganttchart

3.5 Analysis chart of project

3.5.1 Use-Case Diagram (Low Level):

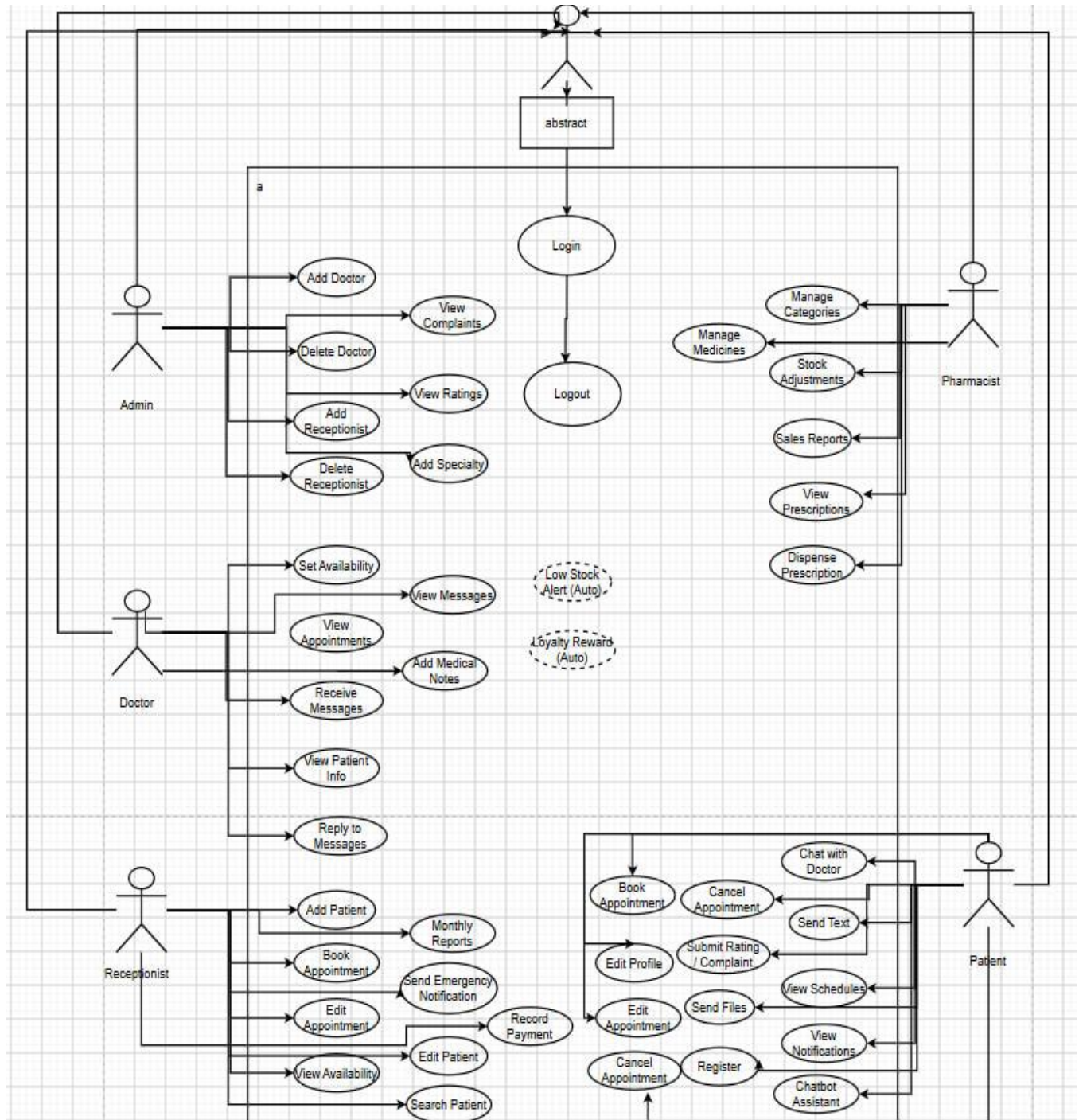


Figure 3 chapter 3.2 use case low level

3.5.2 Use-Case Diagram (High Level):

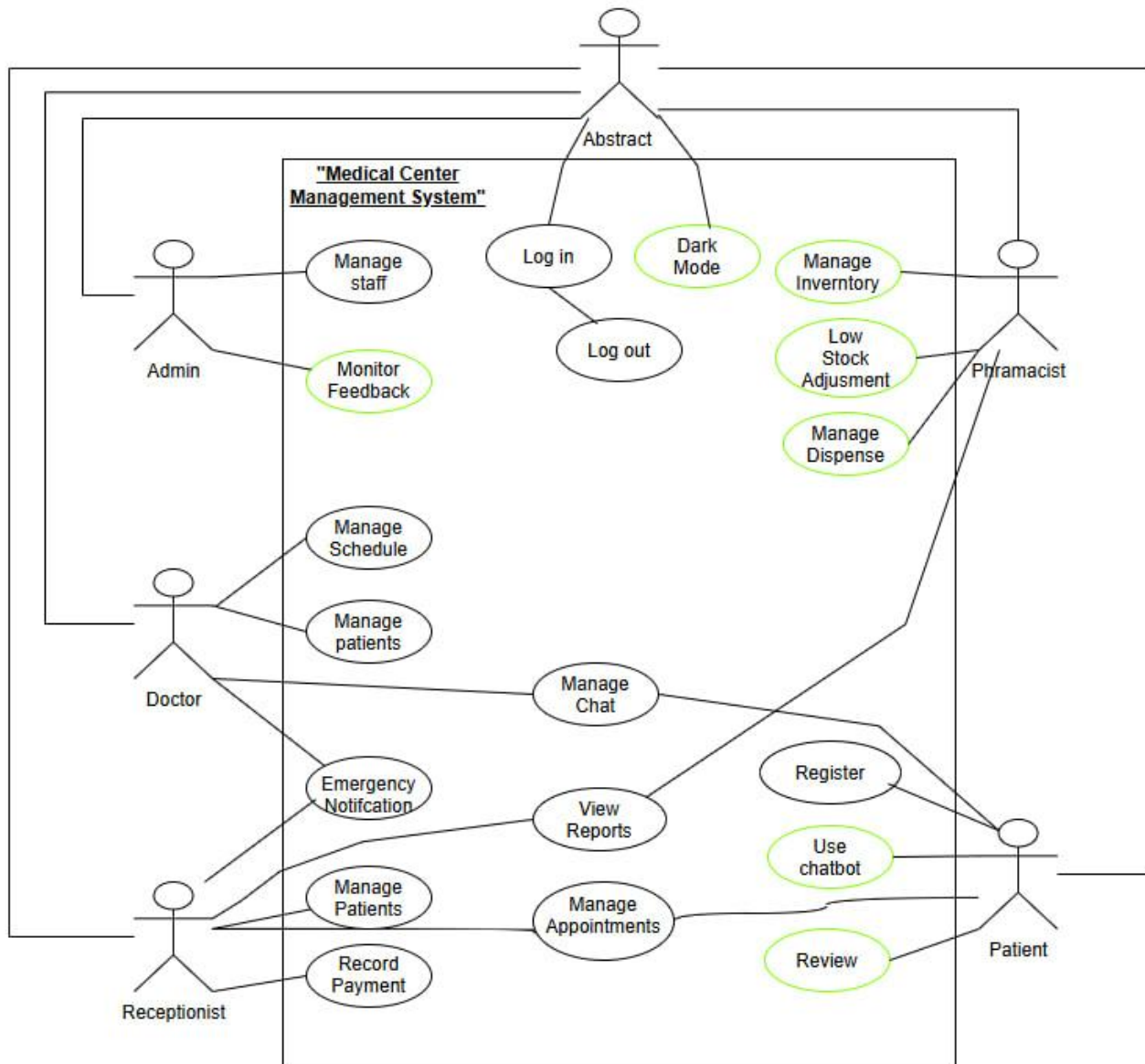


Figure 4 chapter3.3. use case high level

3.5.3 Activity diagram (add category)

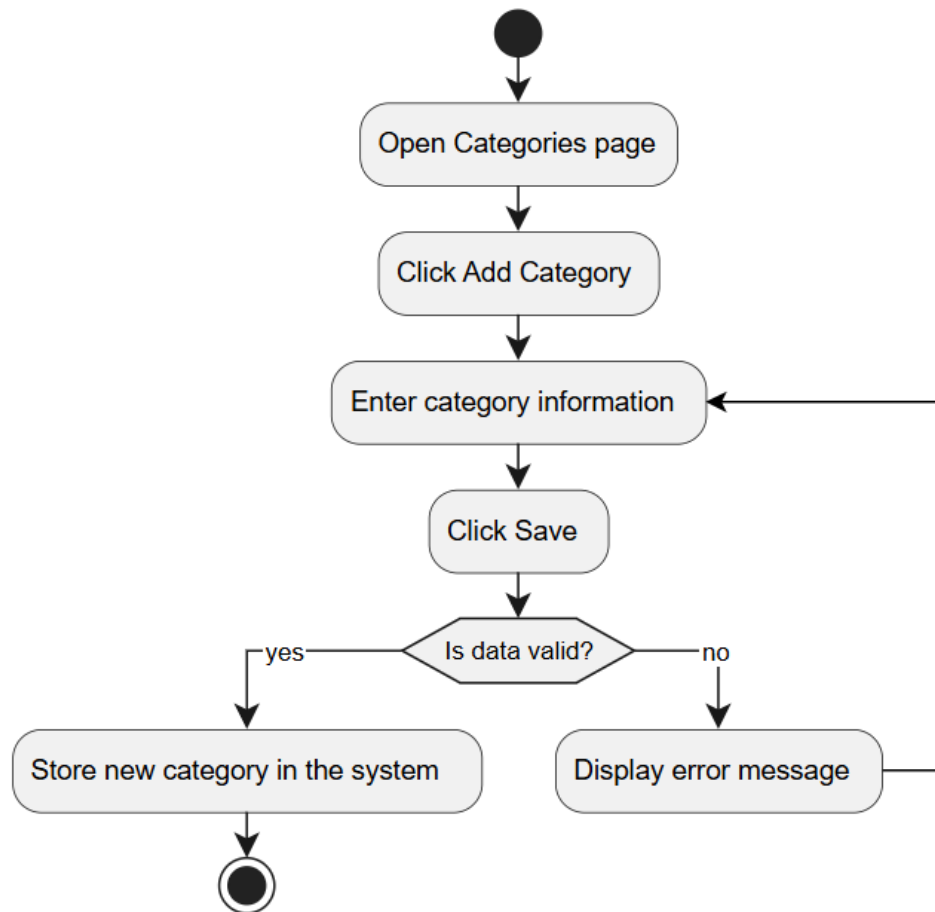


Figure 5 chapter3.5 ad add category

3.5.4 Activity Diagram (edit medicine diagram)

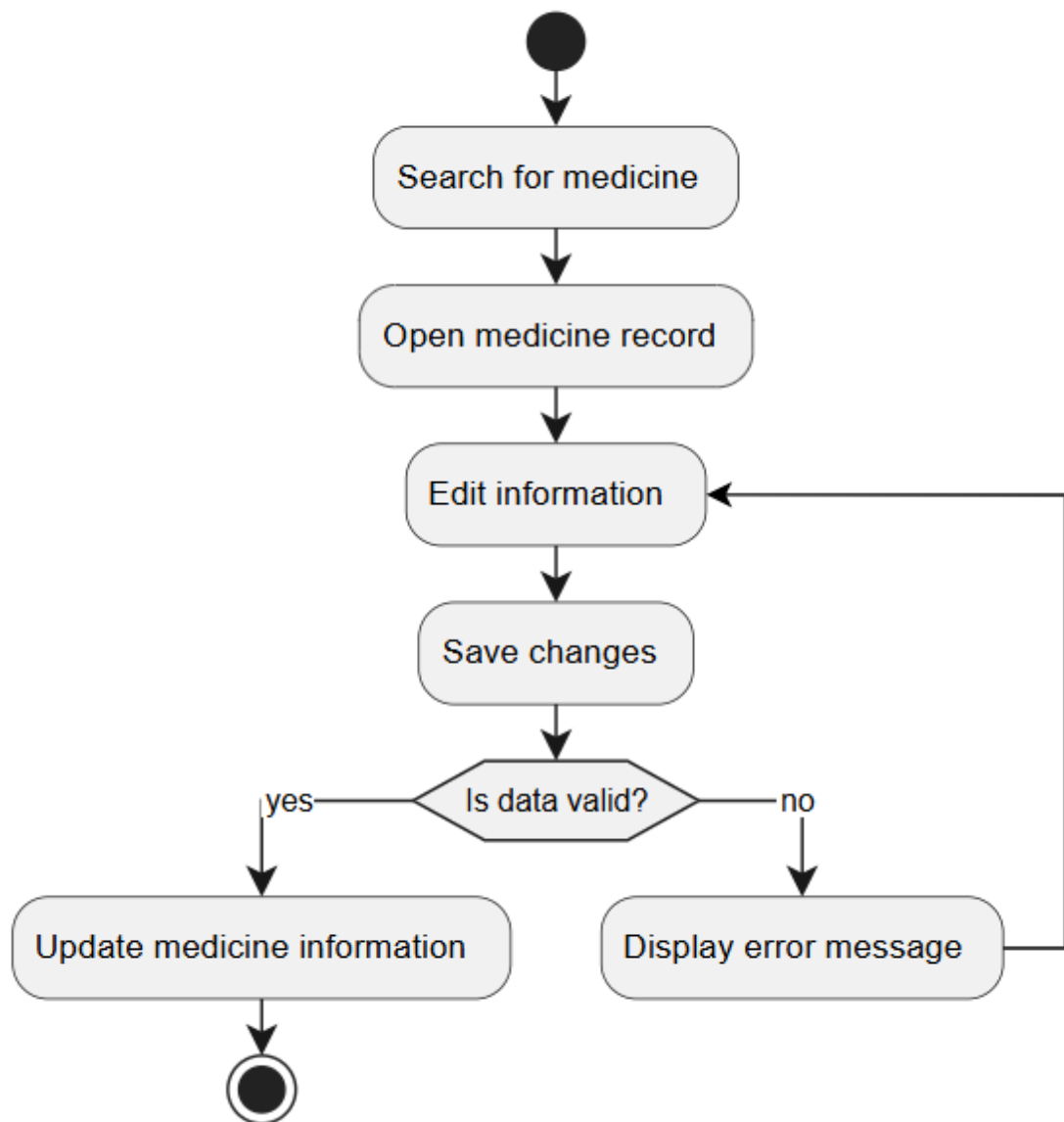


Figure 6 chapter3.6 ad edit medicine diagram

3.5.5 Activity diagram (dispense diagram)

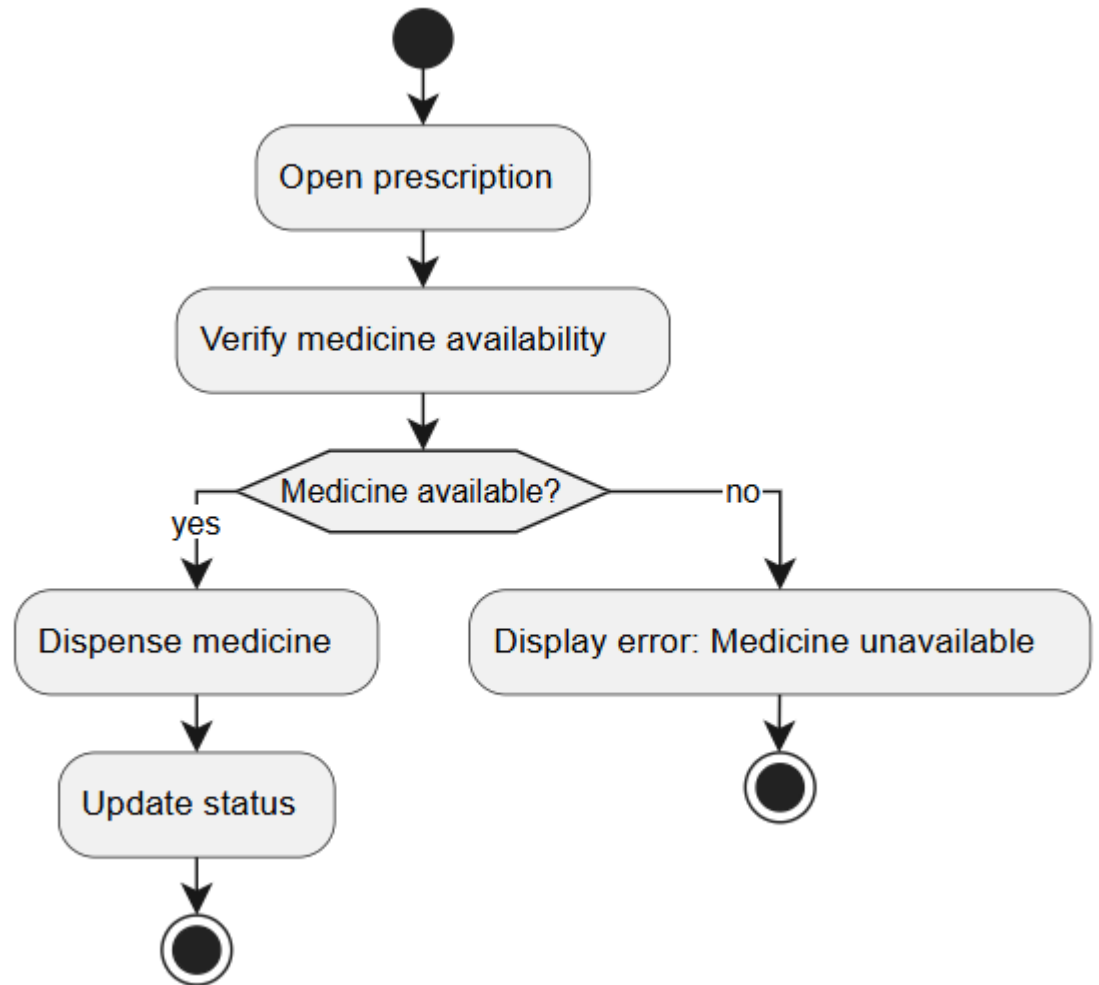


Figure 7 chapter3.7 ad dispense diagram

3.5.6 Activity diagram (view available medicines)

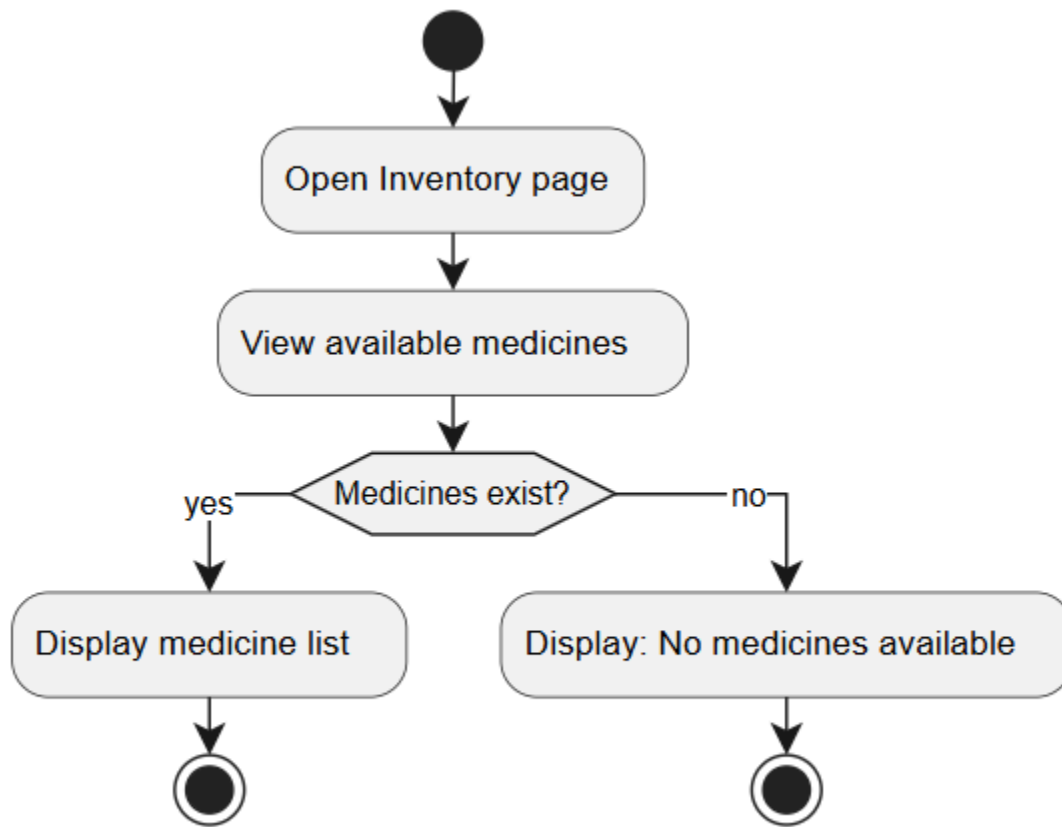


Figure 8 chapter3. 8 ad view available medicines

3.5.7 Activity diagram (view daily sales)

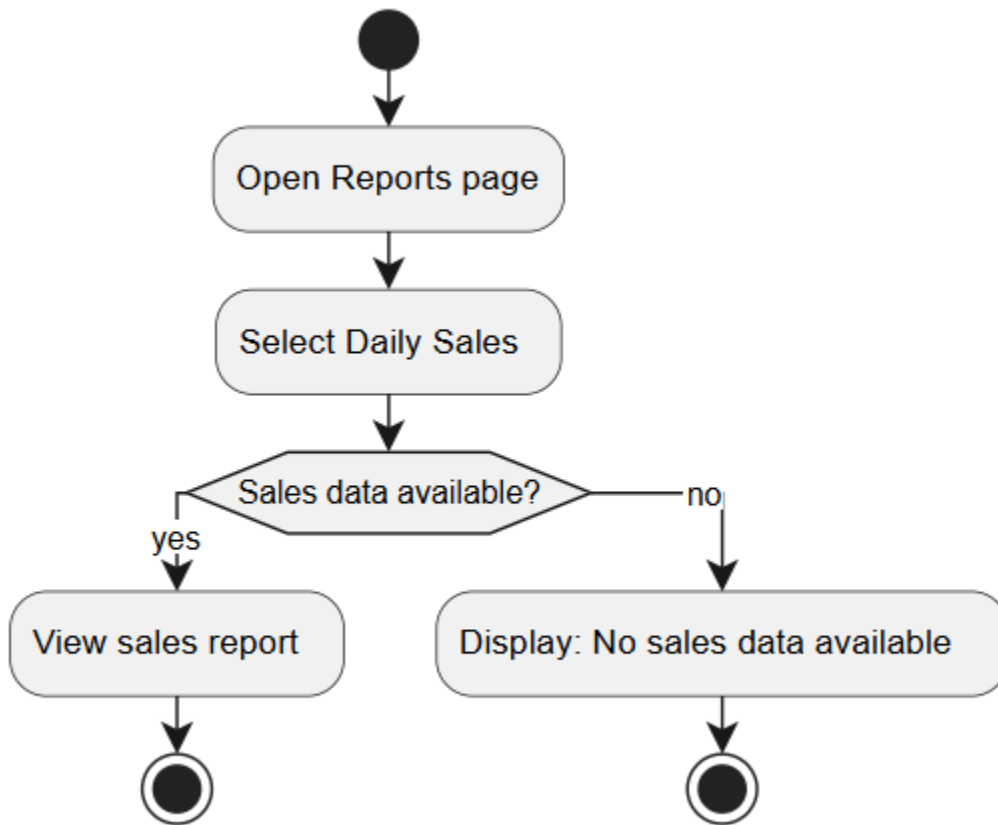


Figure 9 chapter3.9 ad view daily sales

3.5.8 Activity diagram (assign category)

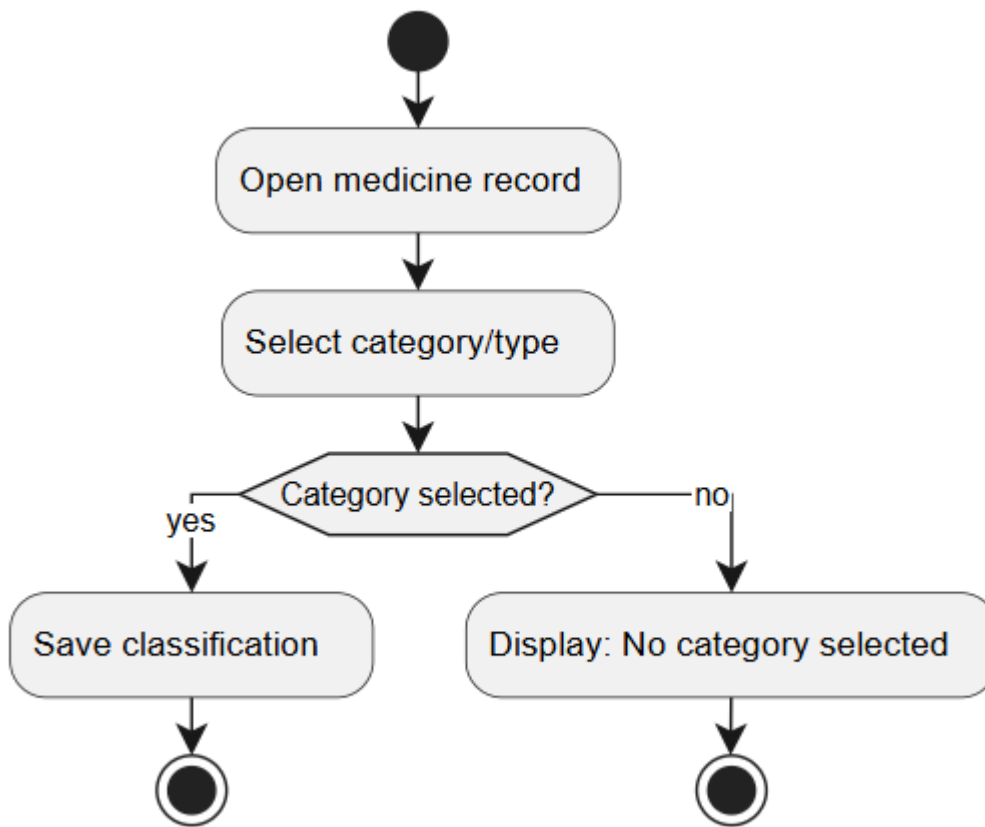


Figure 10 chapter3.10 ad assign category

3.5.9 Activity diagram (display faqs)

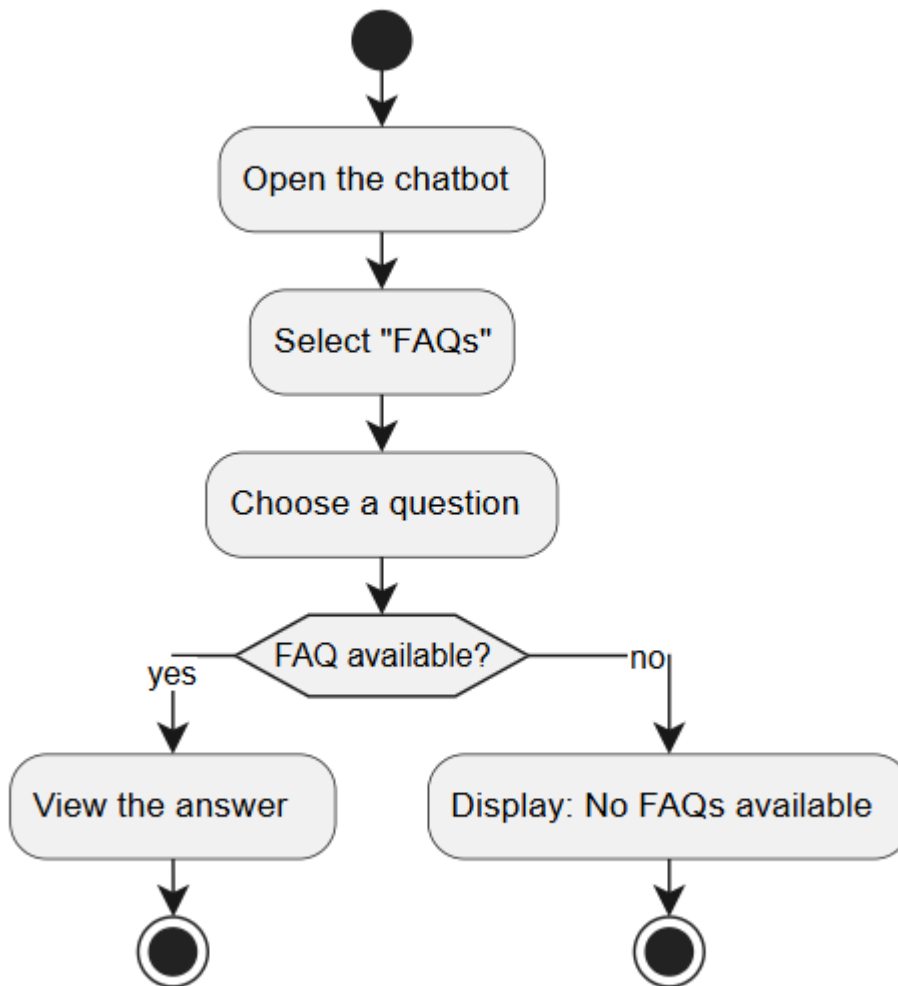


Figure 11 chapter3.11 ad display faqss

3.5.10 Activity Diagram (view complaints)

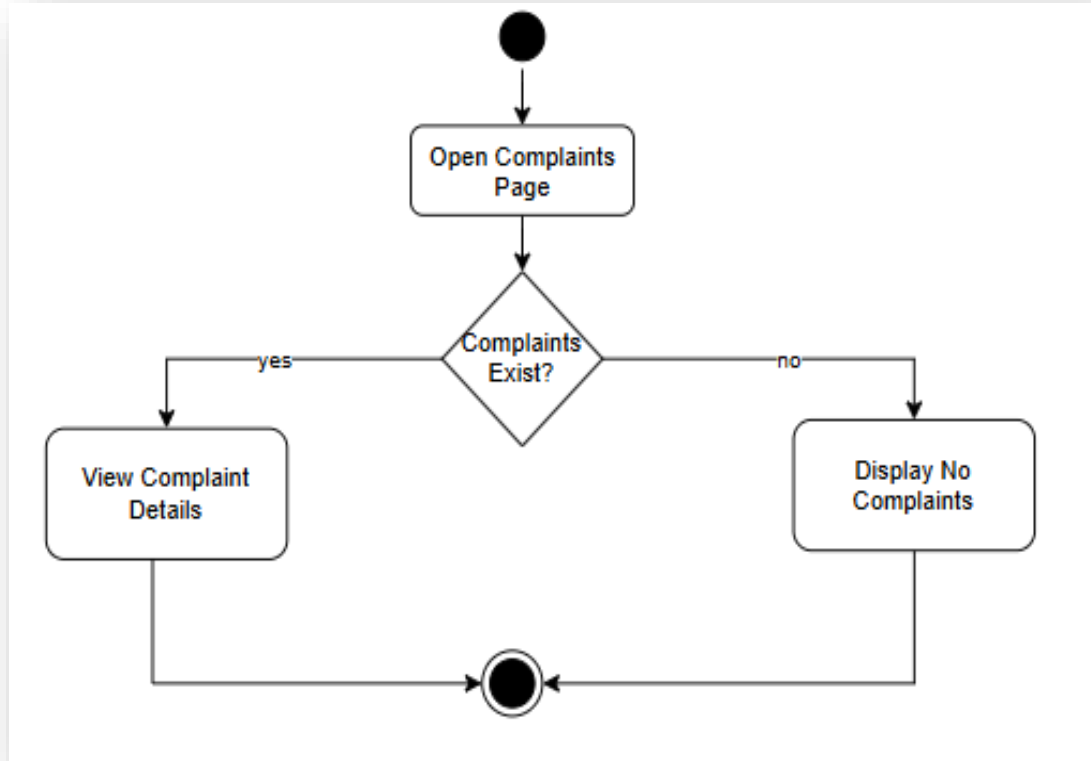


Figure 12 chapter3.12 ad view complaints

3.5.11 Activity diagram (rate a doctor)

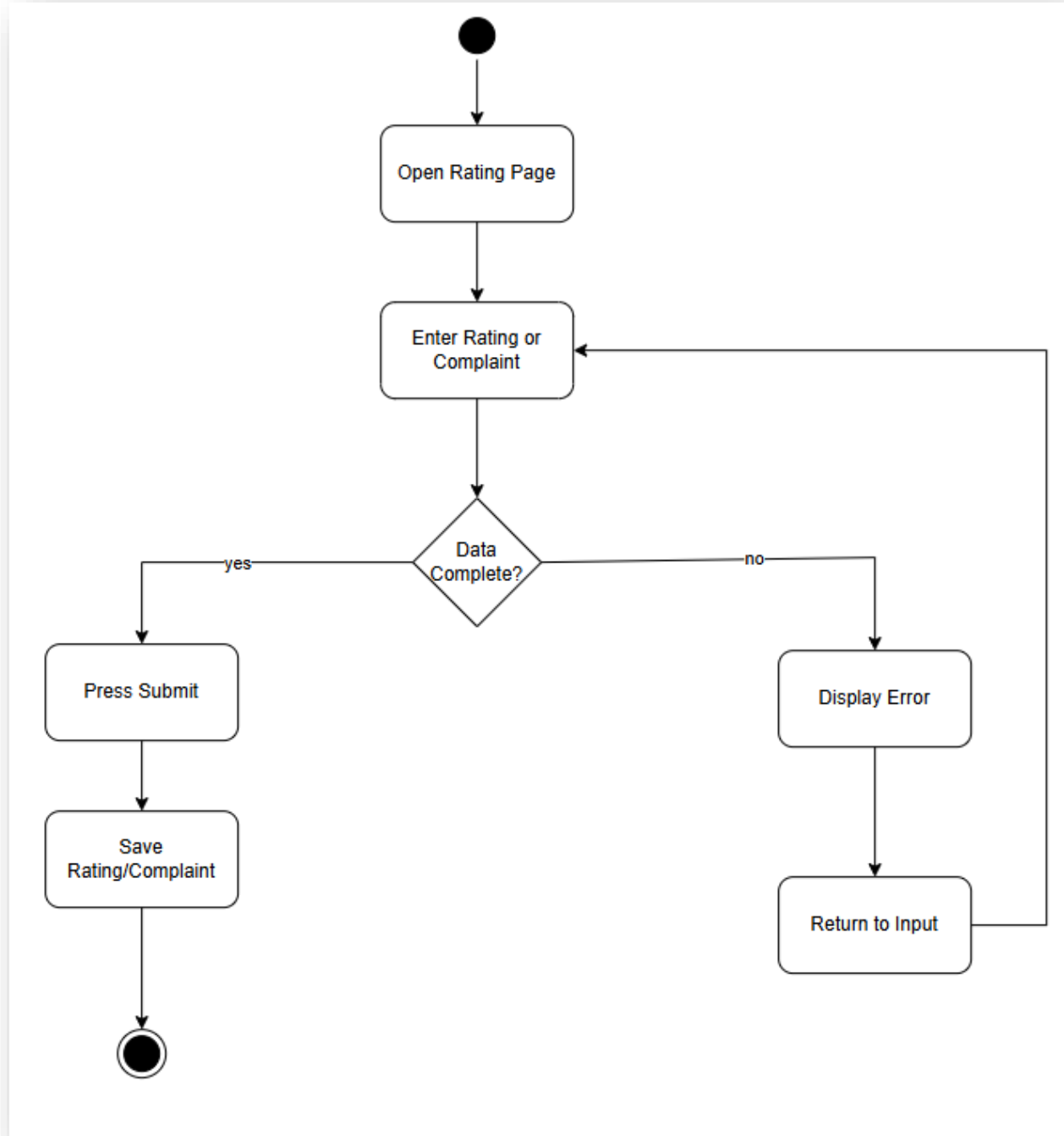


Figure 13 chapter3.13 ad rate a doctor

3.5.12 Activity diagram (add specialty)

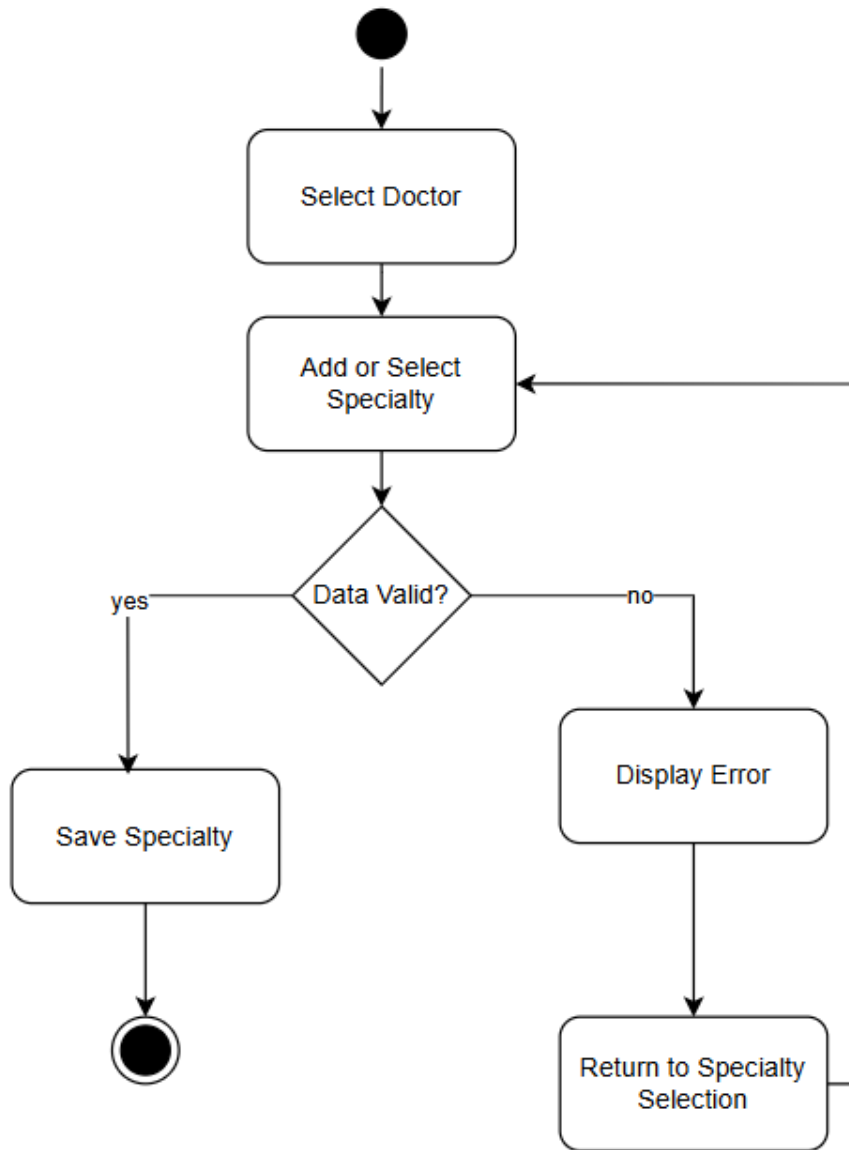


Figure 14 chapter3.14 ad add specialty

3.5.13 Sequence diagram (log in)

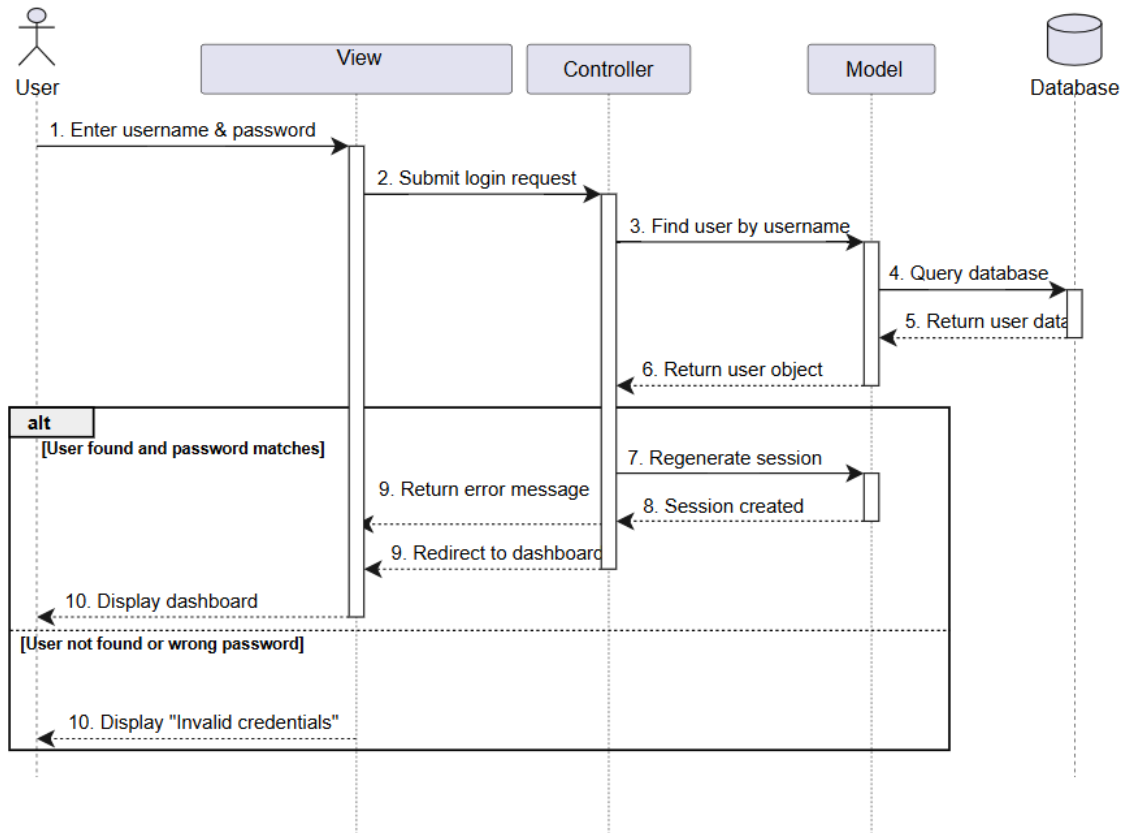


Figure 15 chapter3.15 SQ login

3.5.14 Sequence diagram (Add category) ...

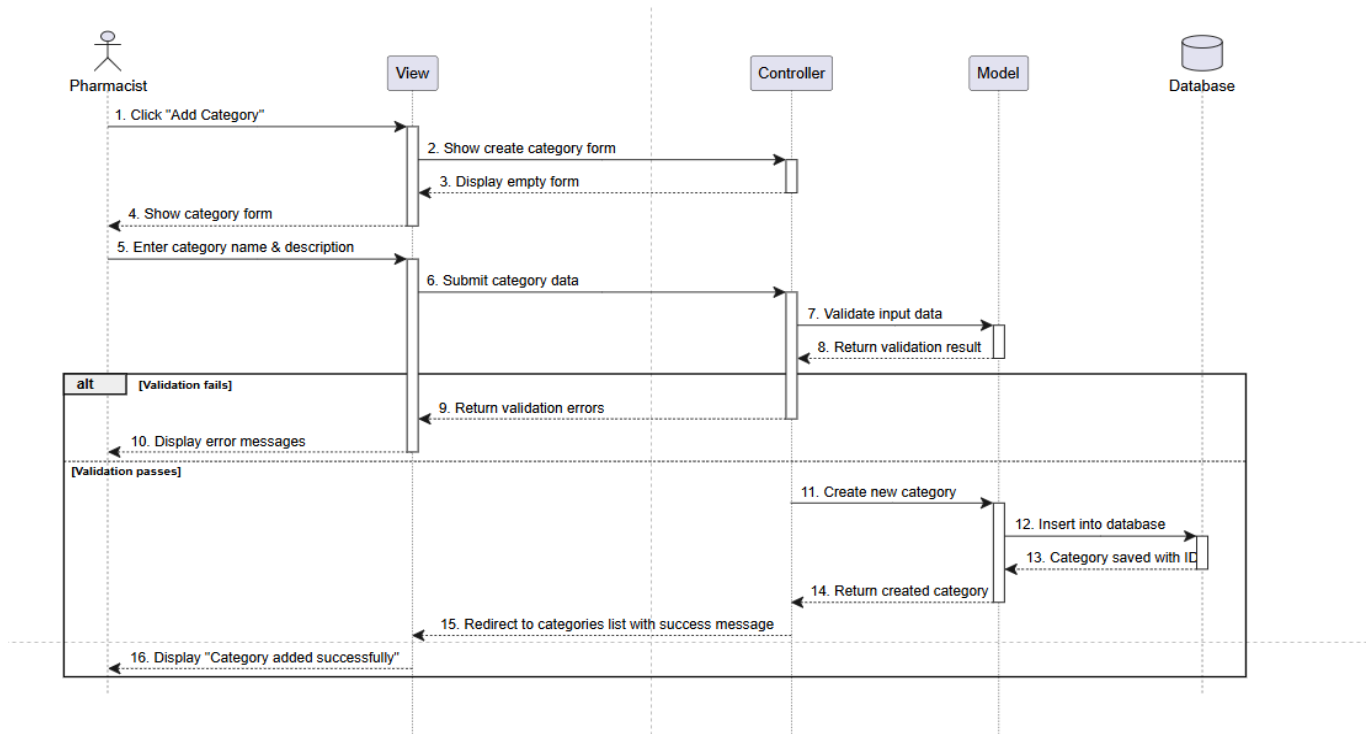


Figure 16 chapter3.16 sq add category

3.5.15 Sequence Diagram (Add Pharmacist)

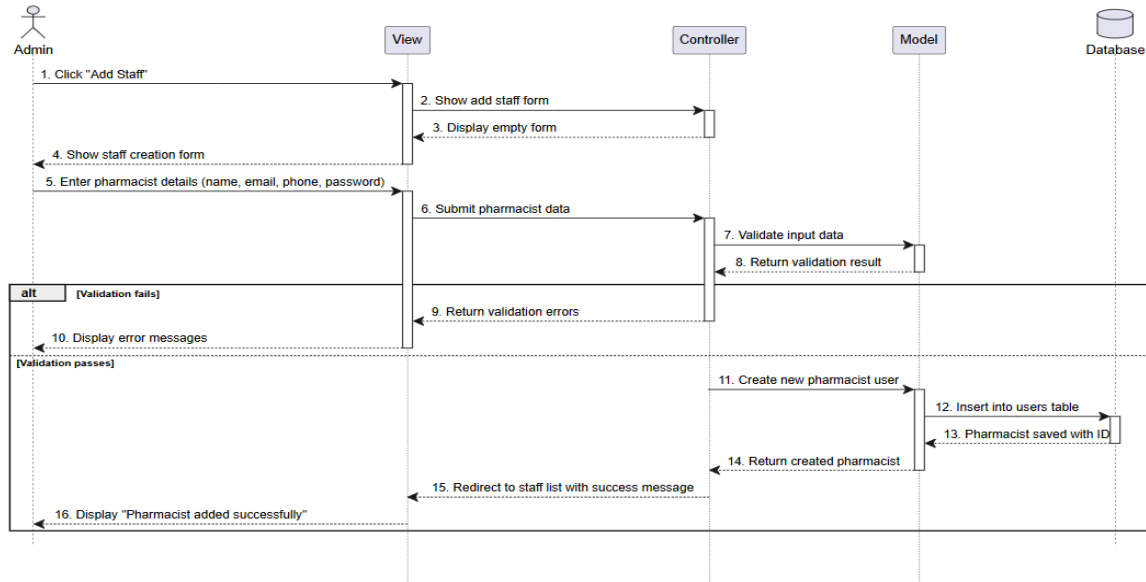


Figure 17 chapter3 3.17 sq add staff

3.5.16 Sequence Diagram (Edit medicine Info)

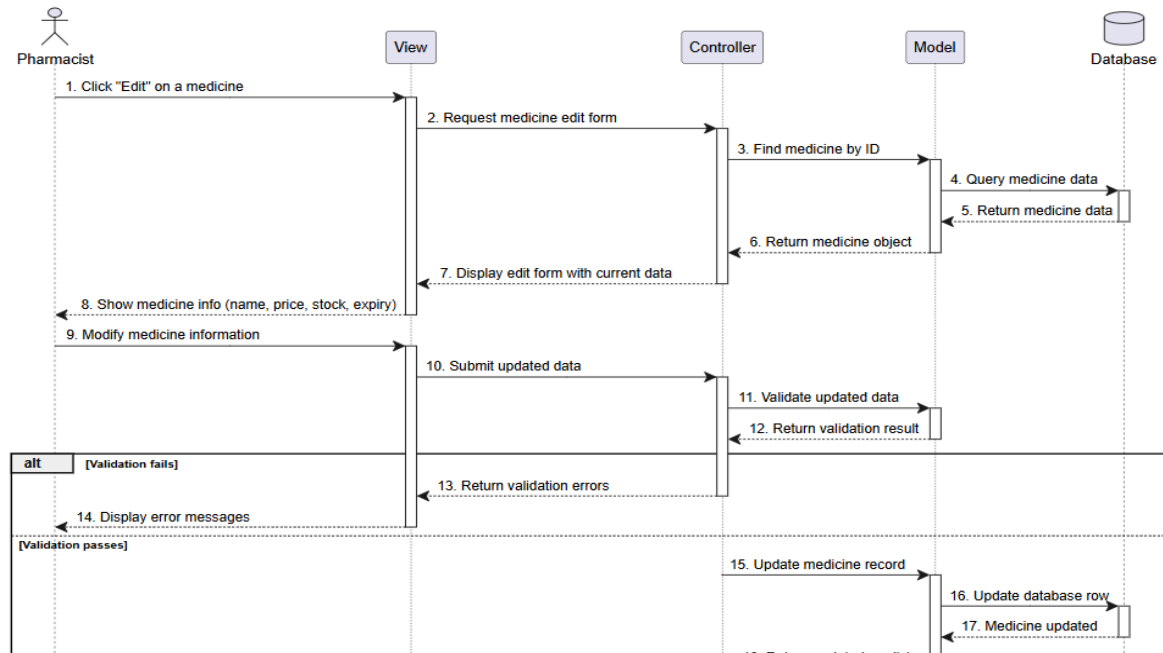


Figure 18 chapter3.18 sq edit medicine info

3.5.17 Sequence Diagram (Despines Medicine)

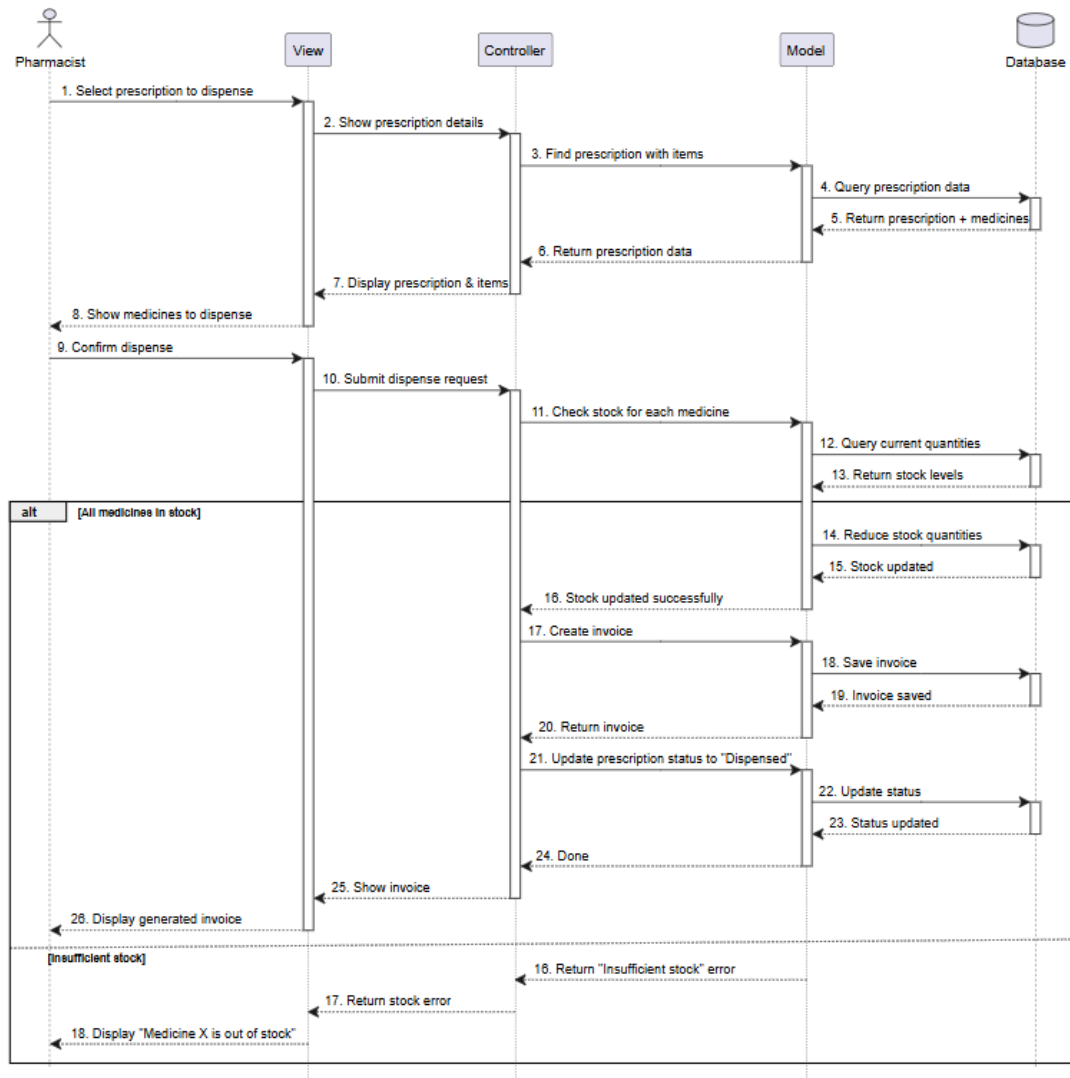


Figure 19 chapter3.19 sq dispince medicine

3.5.18 Sequence Diagram (Notify Low medicine)

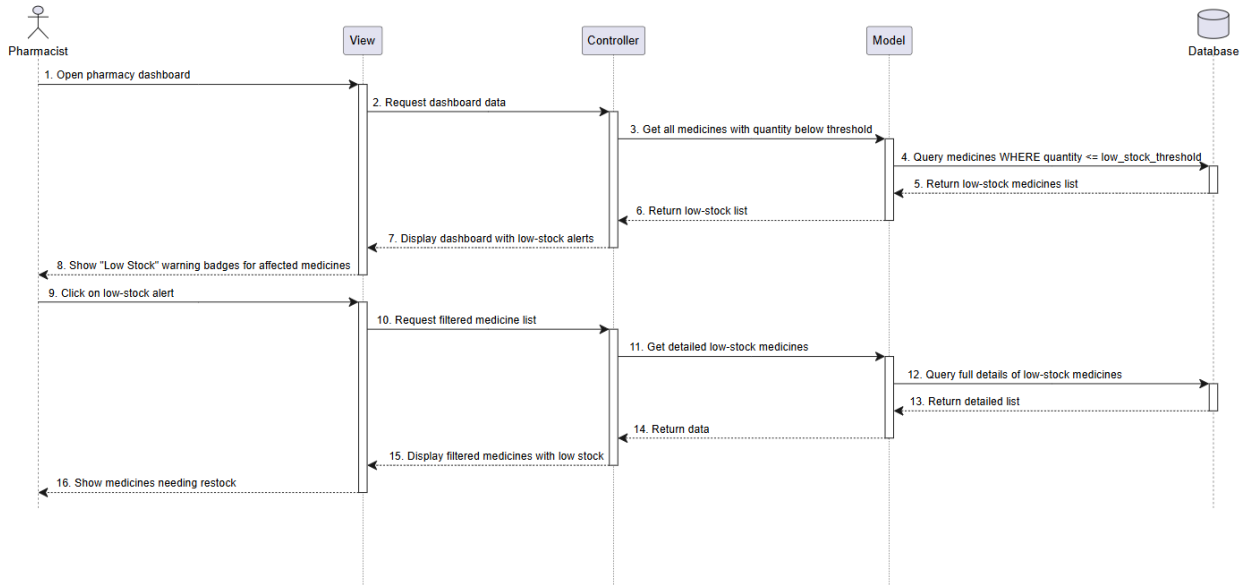


Figure 20 chapter3.20 sq notify low medicine

3.5.19 Sequence Diagram(View Available Medicine)

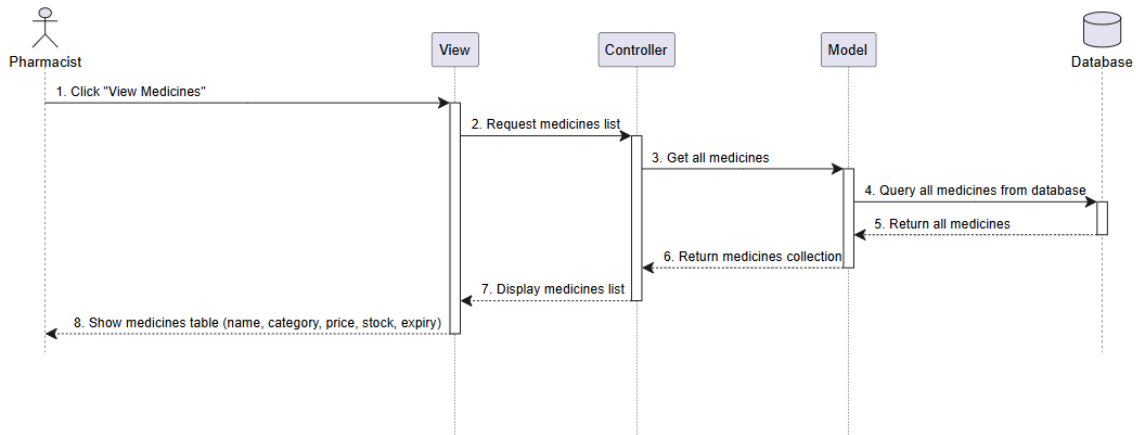


Figure 21 chapter3.21 sq View Available Medicine

3.5.20 Sequence Diagram (View Daily Sale Record)

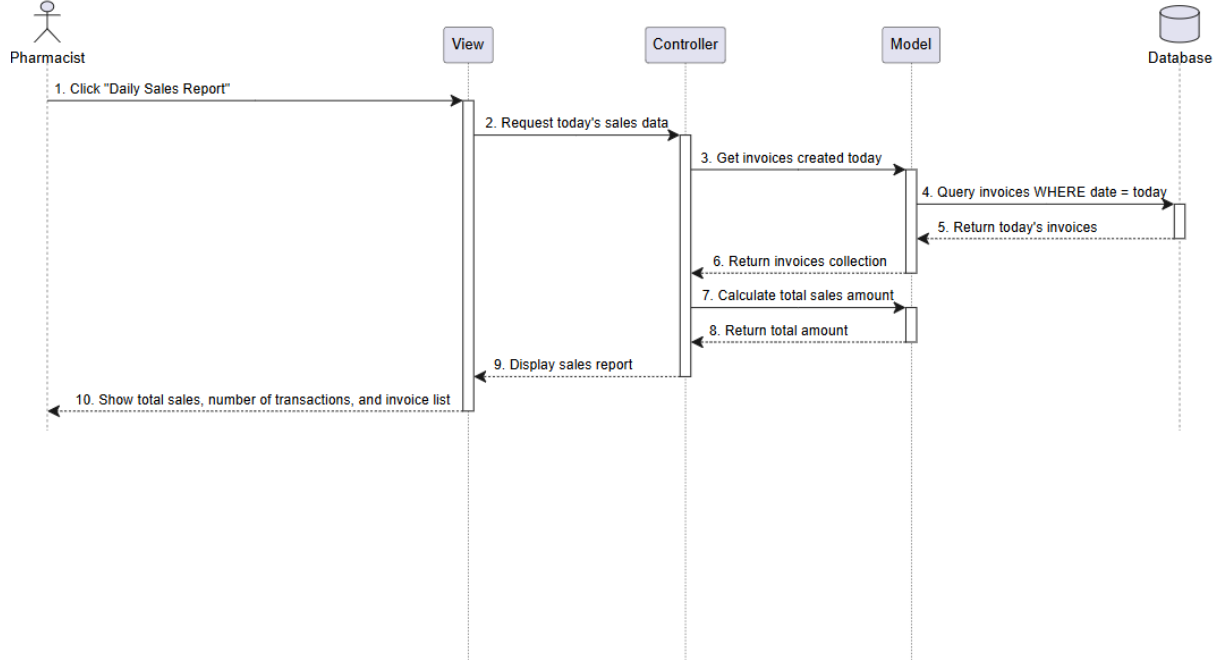


Figure 22 chapter3.22 sq View Daily Sale Record

3.5.21 Sequence Diagram (Assign Category)

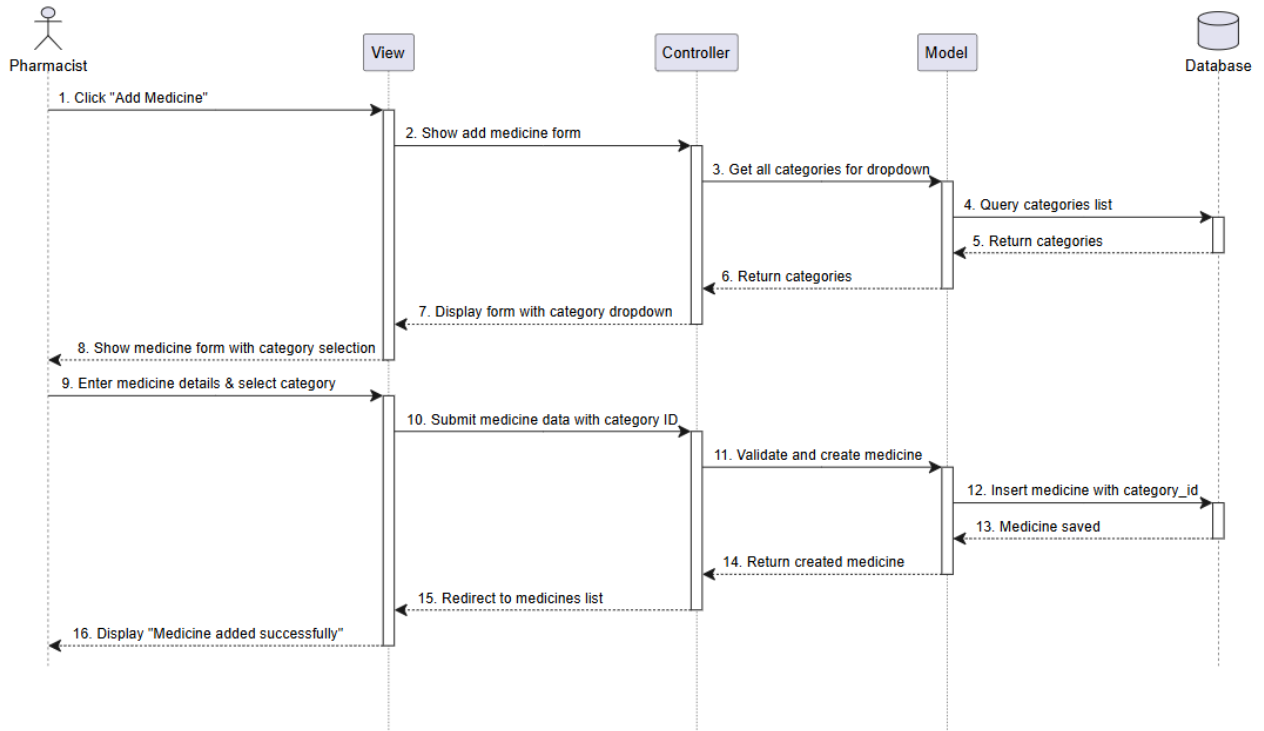


Figure 23 chapter3.23 sq Assign Category

3.5.22 Sequence Diagram(Display FAQs and Answer)

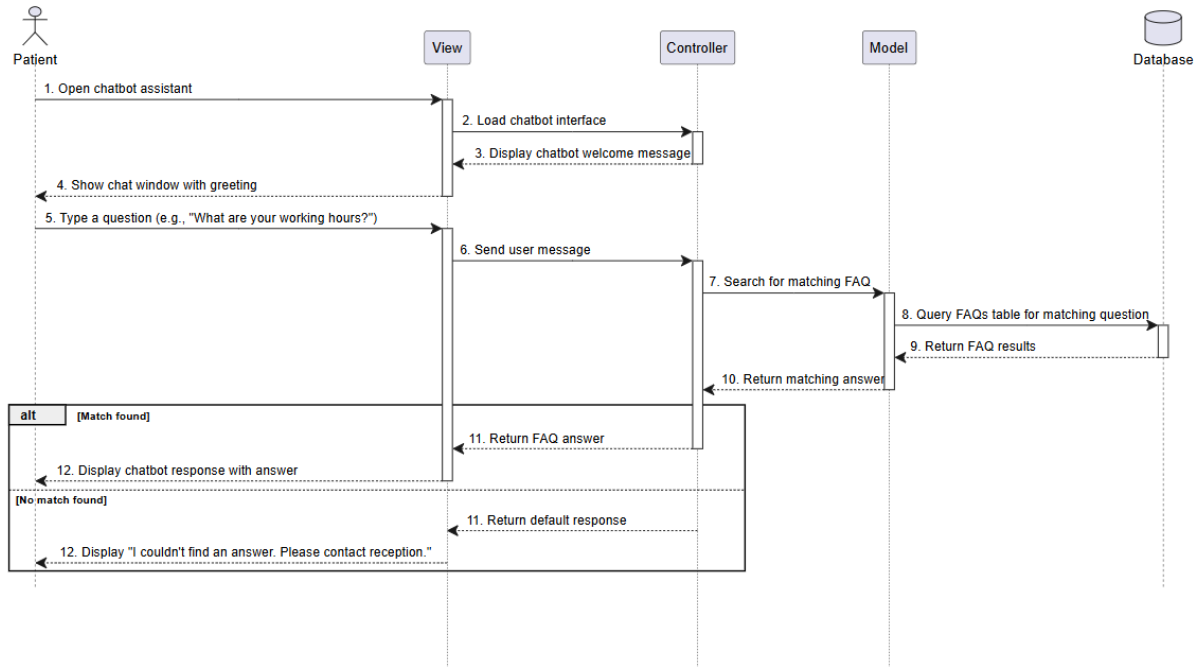


Figure 24 chapter3.24 sq Display FAQs and Answer

3.5.23 Sequence Diagram (Explain Appointment Booking Process)

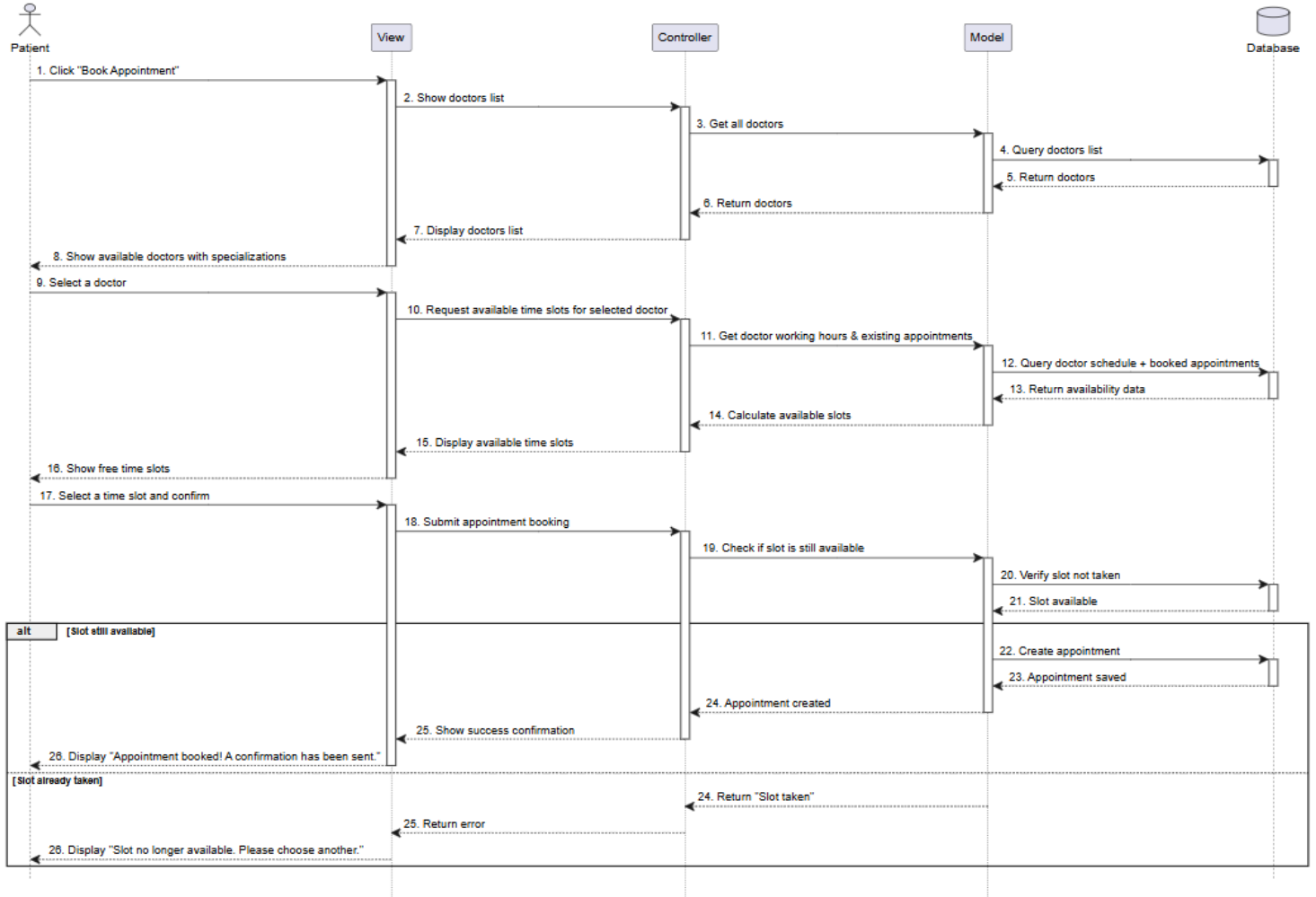


Figure 25 chapter3.25 sq Explain Appointment Booking Process

3.5.24 Sequence Diagram (Patient Chat with Doctor)

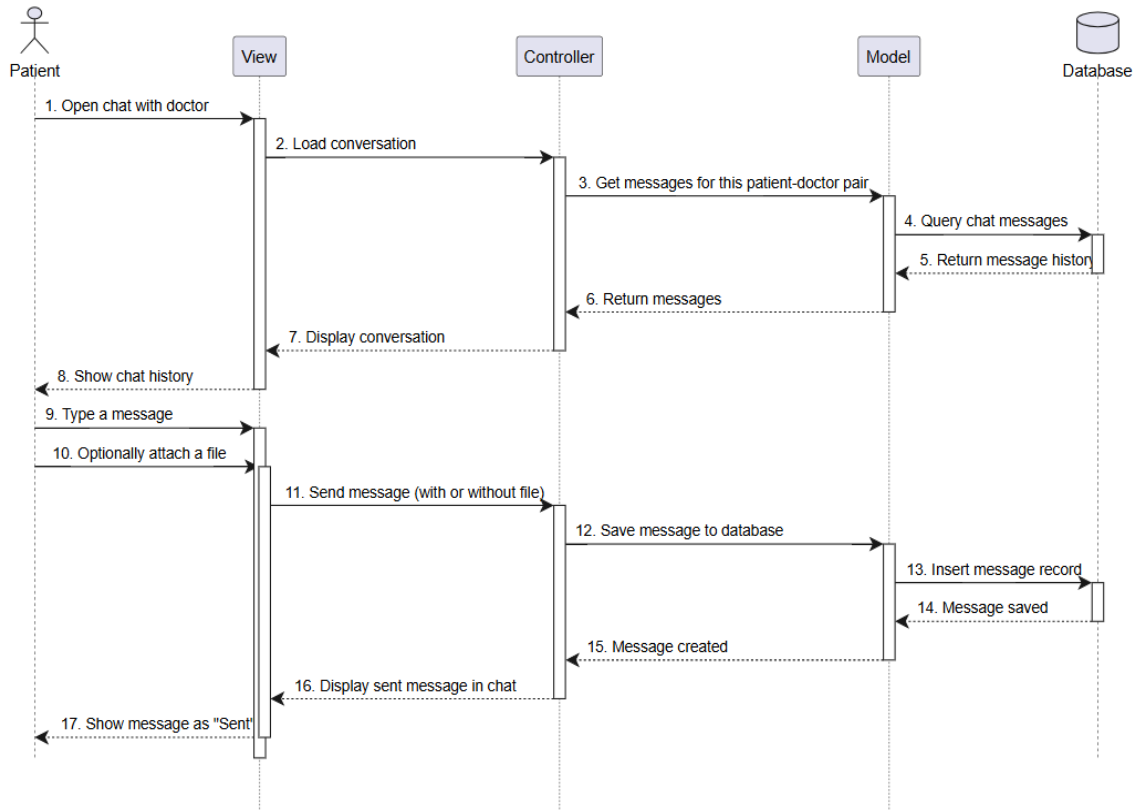


Figure 26 chapter3.26 sq Patient Chat with Doctor

3.5.25 Sequence Diagram (Send Emergency Notification)

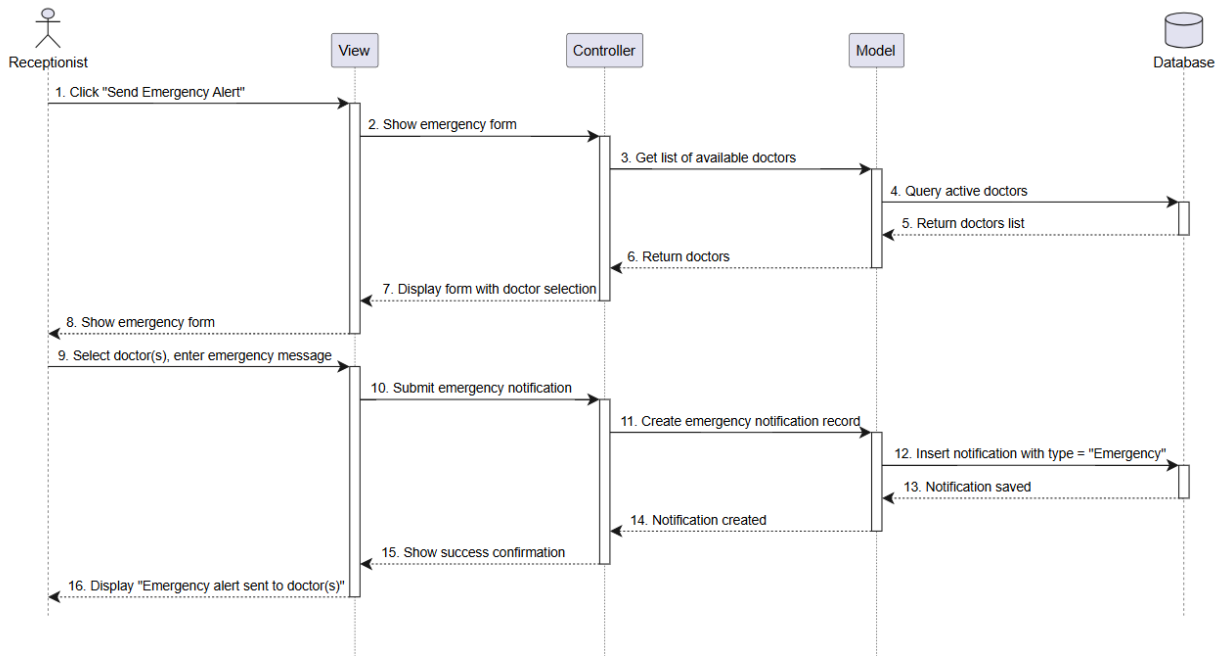


Figure 27 chapter3.27 sq Send Emergency Notification

3.5.26 Sequence Diagram (Patient Rating a Doctor)

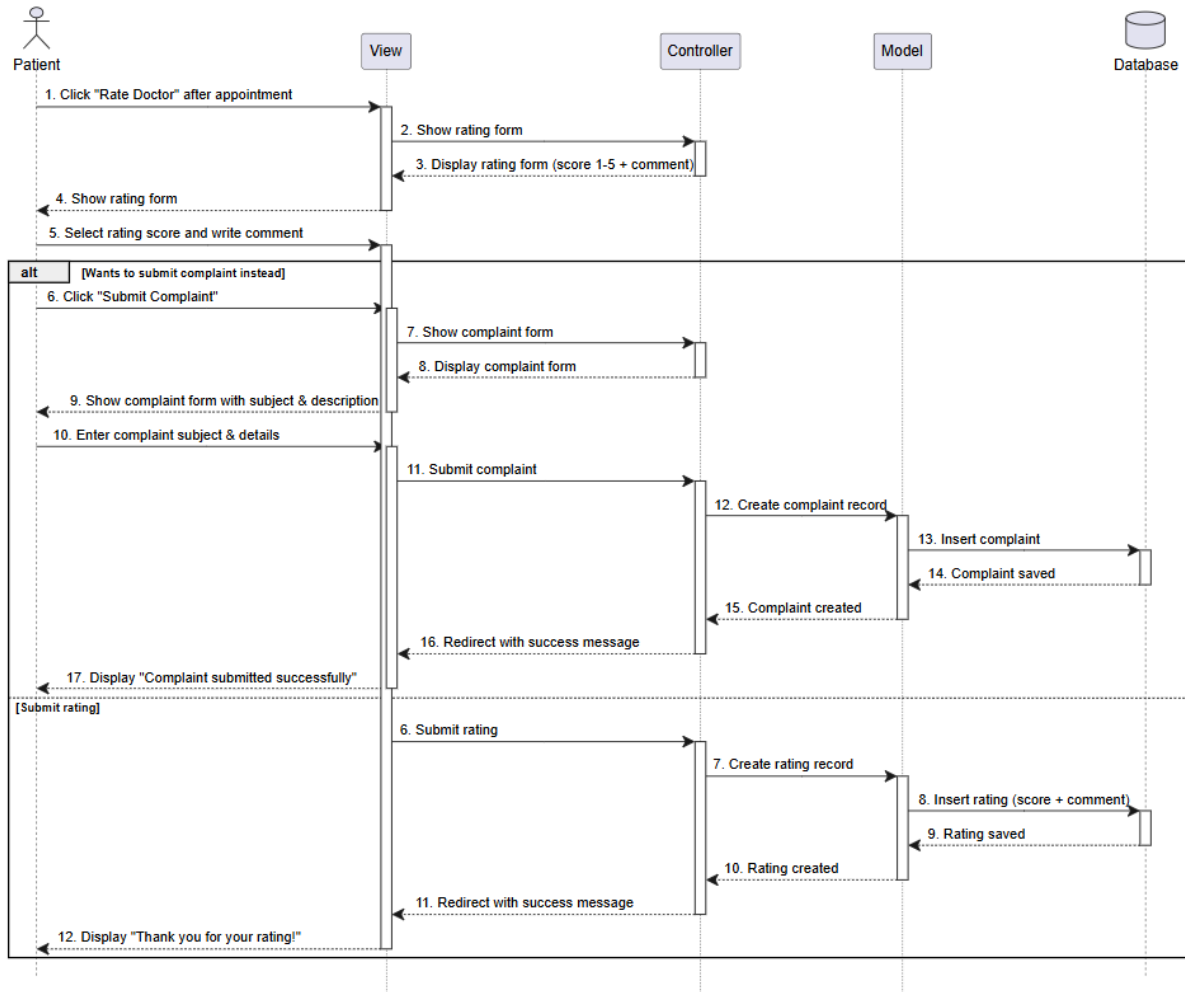


Figure 28 chapter3.28 sq Patient Rating a Doctor

3.5.27 Sequence Diagram (Register a Patient)

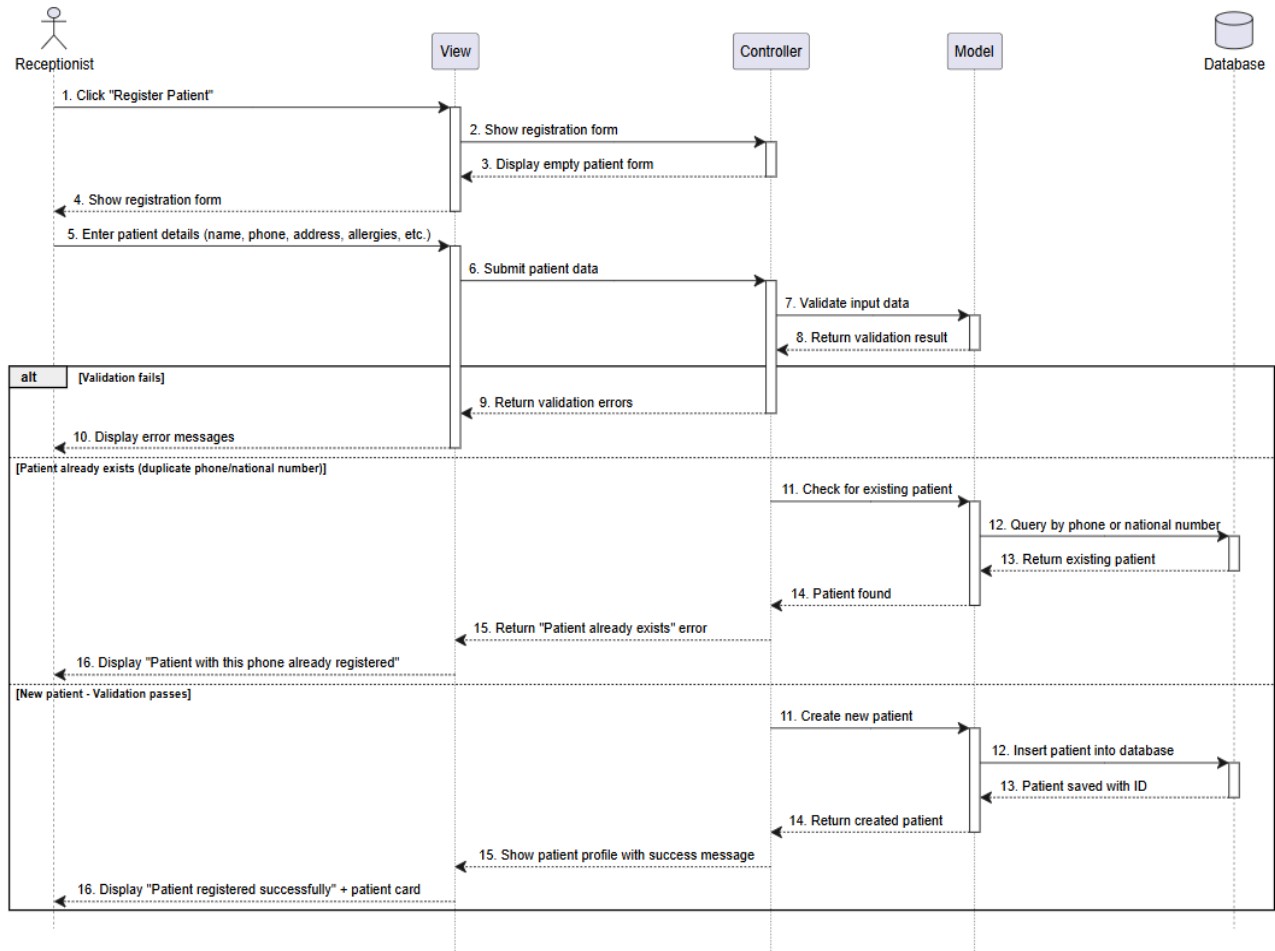


Figure 29 chapter3.29 sq Register a Patient

3.5.28 Sequence Diagram (Doctor Managing Weekly Availability)

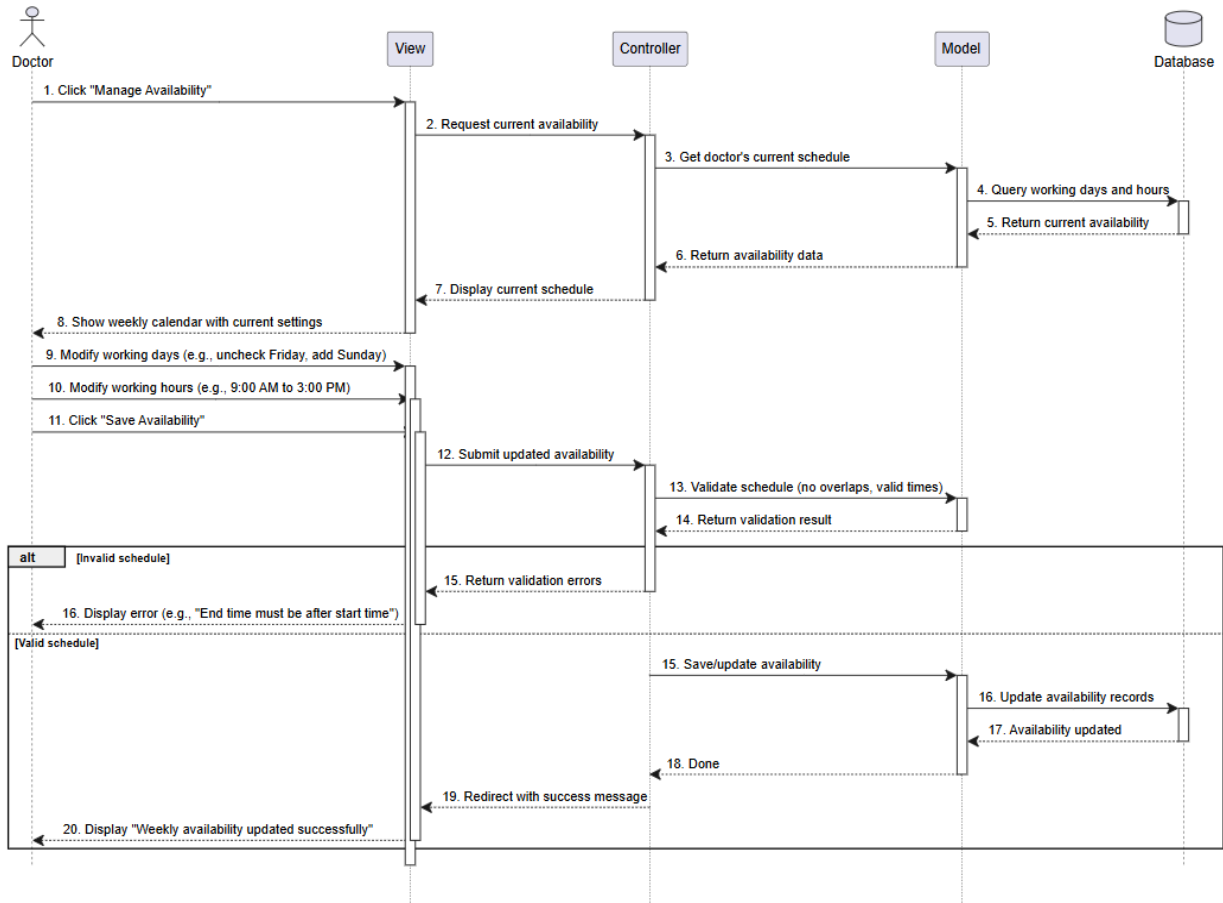


Figure 30 chapter3.30 sq Doctor Managing Weekly Availability

3.5.1 Use case Specification:
3.5.25.1 Login

Table 1 chapter 3.1 log in

Field	Details
Use Case	Login
Use Case ID	UC-01
Actor	Admin, Receptionist, Doctor
Goal	Log into the system using email and password.
Preconditions	User account exists.
Main Flow	1) Open login page 2) Enter email and password 3) System validates 4) System redirects to dashboard based on role
Alternative Flow	If credentials are wrong show error
Postconditions	User is logged in.

3.5.25.2 Add Doctor

Table 2 chapter 3.2 add doctor

Field	Details
Use Case	Add Doctor
Use Case ID	UC-03
Actor	Admin
Goal	Add a new doctor account.
Preconditions	Admin is logged in.
Main Flow	1) Open staff management 2) Click Add Doctor 3) Enter details 4) Save 5) System creates doctor user
Alternative Flow	Missing or duplicate email shows validation error.
Postconditions	Doctor account is created.

3.5.25.3 Add Receptionist

Table 3 chapter 3.3 add receptionist

Field	Details
Use Case	Add Receptionist
Use Case ID	UC-05
Actor	Admin
Goal	Add a new receptionist account.
Preconditions	Admin is logged in.
Main Flow	1) Open staff management 2) Click Add Receptionist 3) Enter details 4) Save 5) System creates receptionist user
Alternative Flow	None
Postconditions	Receptionist account created.

3.5.25.4 Add New Medicine Category

Table 4 chapter 3.4 add new medicine category

Field	Details
Use Case	Add New Medicine Category
Use Case ID	FR-37
Description	The pharmacist adds a new medicine category to the inventory.
Actor	Pharmacist
Precondition	The category does not already exist.
Postcondition	New category is stored in the system.
Main Scenario	1. Open Categories page. 2. Click Add Category. 3. Enter category information. 4. Click Save.
Alternative Scenario	Invalid data entered, return to step 3.

3.5.25.5 Edit Medicine Information

Table 5 chapter 3.5 edit medicine information

Field	Details
Use Case	Edit Medicine Information
Use Case ID	FR-38
Description	The pharmacist updates medicine information.
Actor	Pharmacist
Precondition	Medicine exists in inventory.
Postcondition	Medicine information is updated.
Main Scenario	1. Search for medicine. 2. Open medicine record. 3. Edit information. 4. Save changes.
Alternative Scenario	Invalid data entered, return to step 3.

3.5.25.6 Delete Medicine Category

Table 6 chapter 3.6 delete medicine category

Field	Details
Use Case	Delete Medicine Category
Use Case ID	FR-39
Description	The pharmacist deletes a medicine category.
Actor	Pharmacist
Precondition	Category exists in the system.
Postcondition	Category is removed.
Main Scenario	1. Open Categories page. 2. Select category. 3. Click Delete. 4. Confirm deletion.
Alternative Scenario	Category contains medicines and cannot be deleted.

3.5.25.7 View Available Medicines

Table 7 chapter 3.7 view available medicines

Field	Details
Use Case	View Available Medicines
Use Case ID	FR-40
Description	The pharmacist views available medicines.
Actor	Pharmacist
Precondition	Medicines exist in inventory.
Postcondition	Medicine list is displayed.
Main Scenario	1. Open Inventory page. 2. View available medicines.
Alternative Scenario	No medicines available.

3.5.25.8 Search Medicine by Name or Code

Table 8 chapter 3.8 search by name or code

Field	Details
Use Case	Search Medicine by Name or Code
Use Case ID	FR-41
Description	Search for a medicine using its name or code.
Actor	Pharmacist
Precondition	Medicine records exist.
Postcondition	Search results are displayed.
Main Scenario	1. Open Search page. 2. Enter medicine name/code. 3. Click Search. 4. View results.
Alternative Scenario	No matching medicine found.

3.5.25.9 Receive Prescription from Doctor

Table 9 chapter 3.9 receive prescription from doctor

Field	Details
Use Case	Receive Prescription from Doctor
Use Case ID	FR-42
Description	The pharmacist receives prescriptions submitted by doctors.
Actor	Pharmacist
Precondition	Doctor has submitted a prescription.
Postcondition	Prescription appears in pending list.
Main Scenario	1. Open Prescriptions page. 2. View received prescriptions.
Alternative Scenario	No prescriptions available.

3.5.25.10 Dispense Medicine

Table 10 chapter 3.10 dispense medicine

Field	Details
Use Case	Dispense Medicine
Use Case ID	FR-43
Description	Pharmacist dispenses medicine and updates status.
Actor	Pharmacist
Precondition	Valid prescription exists.
Postcondition	Prescription marked as Dispensed.
Main Scenario	1. Open prescription. 2. Verify medicine availability. 3. Dispense medicine. 4. Update status.
Alternative Scenario	Medicine unavailable.

3.5.25.11 Notify Low Medicine Stock

Table 11 chapter 3.11 notify low medicine stock

Field	Details
Use Case	Notify Low Medicine Stock
Use Case ID	FR-44
Description	System alerts pharmacist about low stock.
Actor	System
Precondition	Quantity reaches minimum threshold.
Postcondition	Alert notification is displayed.
Main Scenario	1. Monitor inventory. 2. Detect low quantity. 3. Generate notification.
Alternative Scenario	Stock level remains sufficient.

3.5.25.12 Record Medicine Expiration Dates

Table 12 chapter 3.12 record medicine expiration dates

Field	Details
Use Case	Record Medicine Expiration Dates
Use Case ID	FR-45
Description	Store expiration dates for medicines.
Actor	Pharmacist
Precondition	Medicine exists.
Postcondition	Expiration date saved.
Main Scenario	1. Open medicine record. 2. Enter expiration date. 3. Save record.
Alternative Scenario	Invalid date entered.

3.5.25.13 View Daily Sales Record

Table 13 chapter 3.13 view daily sales record

Field	Details
Use Case	View Daily Sales Record
Use Case ID	FR-46
Description	View daily pharmacy sales.
Actor	Pharmacist
Precondition	Sales records exist.
Postcondition	Sales report displayed.
Main Scenario	1. Open Reports page. 2. Select Daily Sales. 3. View report.
Alternative Scenario	No sales data available.

3.6 Entity Relationship Diagram (ERD) Low Level :

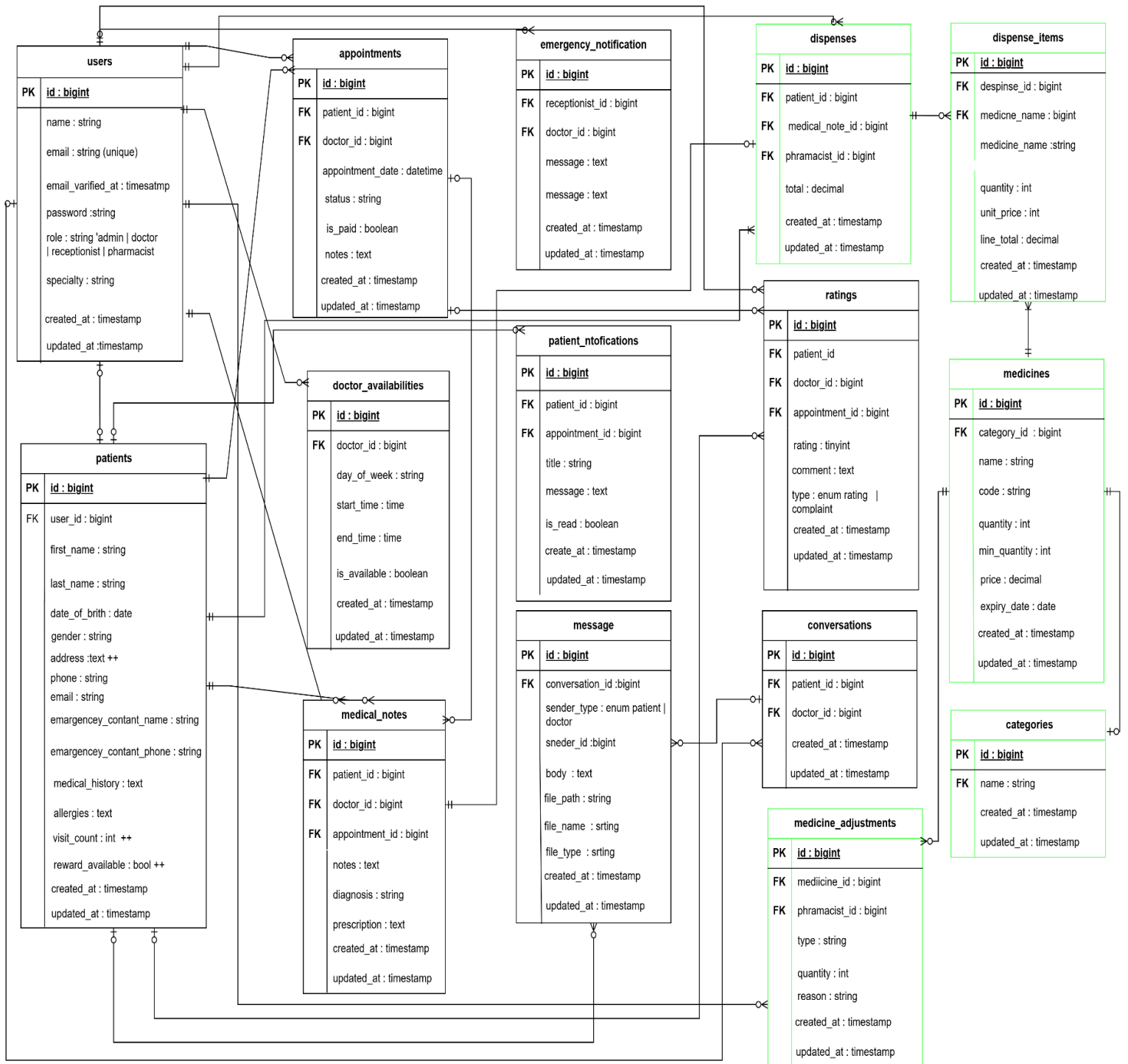


Figure 31 chapter3.31 ERD low level

3.7 Entity Relationship Diagram (ERD) High Level :

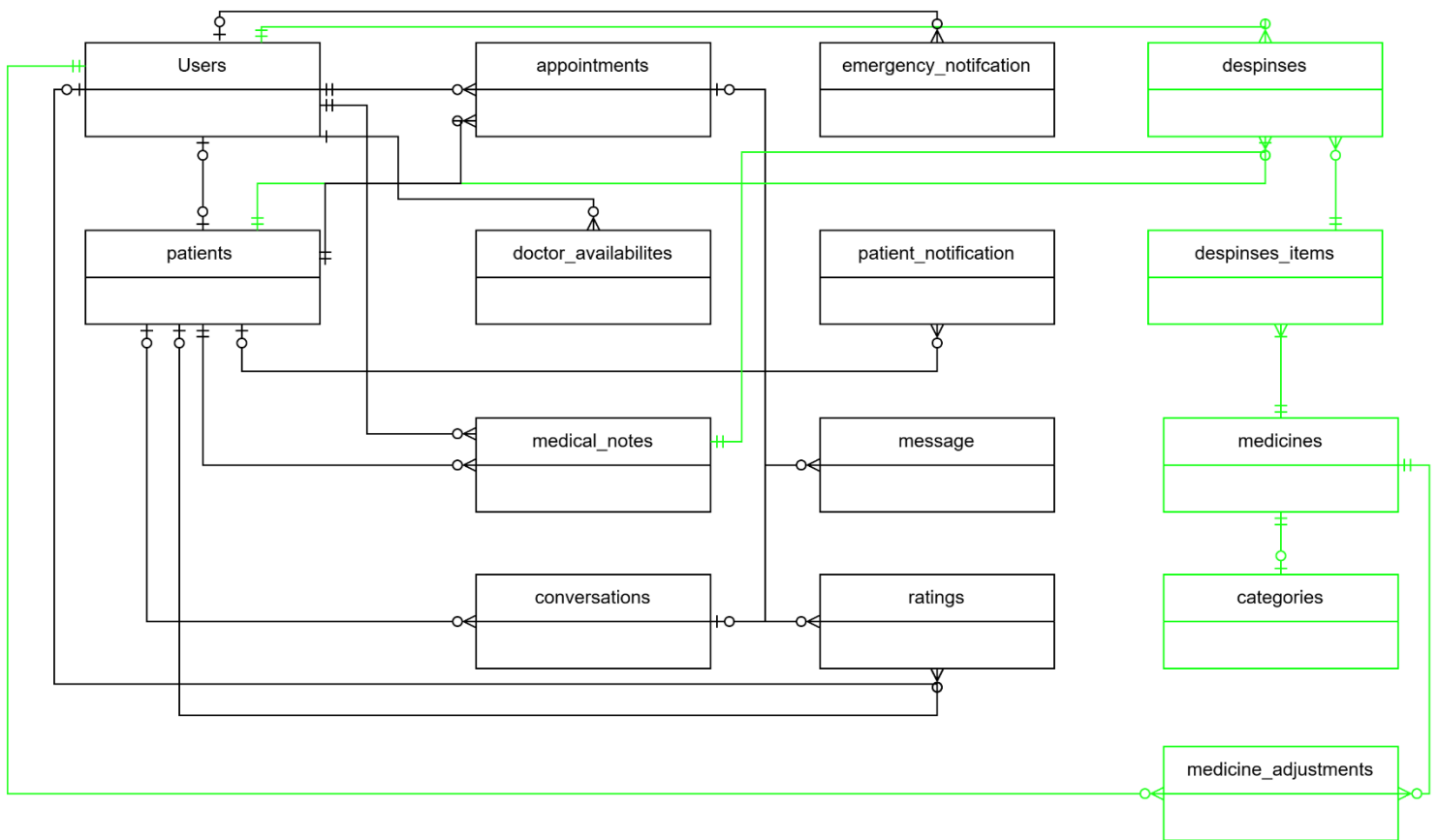


Figure 32 chapter3.32 ERD high level

3.7 Database Tables:

3.7.1 Table Structure for Admin:

Table 14 chapter 3.14 Structure for Admin

Column	Data Type	Index	Description
Id	Int	Primary Key	Admin ID
Username	Varchar	Not Null	Admin Username
Password	Varchar	Not Null	Password for Admin
Name	Varchar	Not Null	Admin Name
Phone Number	Varchar	Not Null	Admin Phone
Address	Varchar	Not Null	Admin Address
Email	Varchar	Not Null	Admin Email

3.7.2 Doctor Table:

Column	Data Type	Index	Description
Id	Int	Primary Key	Doctor ID
Username	Varchar	Not Null	Doctor Username
Password	Varchar	Not Null	Password for Doctor
Name	Varchar	Not Null	Doctor Name
Specialization	Varchar	Not Null	Doctor Specialization
Phone Number	Varchar	Not Null	Doctor Phone
Email	Varchar	Not Null	Doctor Email
Working Days	Varchar	Not Null	Days Available (e.g., Sun-Thu)
Working Hours	Varchar	Not Null	Shift Hours
Status	Boolean	Not Null	Active/Inactive

Table 15 chapter 3.15 Doctor Table

3.7.3 Receptionist Table:

Table 16 chapter 3.16 Receptionist Table

Column	Data Type	Index	Description
Id	Int	Primary Key	Receptionist ID
Username	Varchar	Not Null	Receptionist Username
Password	Varchar	Not Null	Password for Receptionist
Name	Varchar	Not Null	Receptionist Name
Phone Number	Varchar	Not Null	Receptionist Phone
Email	Varchar	Not Null	Receptionist Email
Status	Boolean	Not Null	Active/Inactive

3.7.4 Pharmacist Table:

Table 17 chapter 3.17 Pharmacist Table

Column	Data Type	Index	Description
Id	Int	Primary Key	Pharmacist ID
Username	Varchar	Not Null	Pharmacist Username
Password	Varchar	Not Null	Password for Pharmacist
Name	Varchar	Not Null	Pharmacist Name
Phone Number	Varchar	Not Null	Pharmacist Phone
Email	Varchar	Not Null	Pharmacist Email
Status	Boolean	Not Null	Active/Inactive

3.7.5 Patient Table:

Table 18 chapter 3.18 Patient Table

Column	Data Type	Index	Description
Id	Int	Primary Key	Patient ID
Username	Varchar	Not Null	Patient Username
Password	Varchar	Not Null	Password for Patient
Name	Varchar	Not Null	Patient Name
Birthday	Date	Not Null	Patient Date of Birth
Gender	Varchar	Not Null	Patient Gender
Phone Number	Varchar	Not Null	Patient Phone
Email	Varchar	Not Null	Patient Email
Address	Varchar	Not Null	Patient Address
Blood Type	Varchar	Null	Blood Type Info
Allergies	Text	Null	Known Allergies
National Number	Varchar	Not Null	National ID Number

Visit Count	Int	Not Null	Total Completed Visits
-------------	-----	----------	------------------------

3.7.6 Appointment Table:

Table 19 chapter 3.19 Appointment Table

Column	Data Type	Index	Description
Id	Int	Primary Key	Appointment ID
Patient ID	Int	Foreign Key	References Patient ID
Doctor ID	Int	Foreign Key	References Doctor ID
Receptionist ID	Int	Foreign Key	References Receptionist ID
Appointment Date	Date	Not Null	Date of Appointment
Appointment Time	Time	Not Null	Time of Appointment
Status	Varchar	Not Null	Scheduled/Completed/Cancelled
Notes	Text	Null	Additional Notes

3.7.7 Medical Record Table:

Table 20 chapter 3.20 Medical Record Table

Column	Data Type	Index	Description
Id	Int	Primary Key	Record ID
Patient ID	Int	Foreign Key	References Patient ID
Doctor ID	Int	Foreign Key	References Doctor ID
Appointment ID	Int	Foreign Key	References Appointment ID
Diagnosis	Text	Not Null	Doctor's Diagnosis
Notes	Text	Null	Additional Medical Notes
Record Date	DateTime	Not Null	Date and Time of Record

3.7.8 Prescription Table:

Table 21 chapter 3.21 Prescription Table

Column	Data Type	Index	Description
Id	Int	Primary Key	Prescription ID
Medical Record ID	Int	Foreign Key	References Medical Record ID
Patient ID	Int	Foreign Key	References Patient ID
Doctor ID	Int	Foreign Key	References Doctor ID
Prescription Date	DateTime	Not Null	Date of Prescription
Status	Varchar	Not Null	Pending/Dispensed/Cancelled

3.7.9 Medicine Category Table:

Table 22 chapter 3.22 Medicine Category Table

Column	Data Type	Index	Description
Id	Int	Primary Key	Category ID

Name	Varchar	Not Null	Category Name
Description	Text	Null	Category Description

3.8 Test Cases:

Table 23 chapter 3.23 Test Cases

Test ID	Requirement (FR)	Preconditions	Test Steps	Expected Result	Actual Result
TC-29	Create a new patient account (FR-21)	Email hasn't been used before	1) Go to the registration page 2) Fill in name, email, password and phone number 3) Click Register	The account is created and the user gets logged in right away	Worked as expected the account was created and got redirected to the patient dashboard
TC-30	Register with invalid info (FR-21)	None	1) Open the registration page 2) Use an email that already exists, or leave a	An error message shows up and the account is not created	Worked as expected the validation message was shown

Test ID	Requirement (FR)	Preconditions	Test Steps	Expected Result	Actual Result
			required field blank 3) Click Register		
TC-31	Update patient profile info (FR-22)	Patient is signed in	1) Open the profile page 2) Change the name or phone number 3) Hit Save	The new details are saved with no issues	Worked as expected the info got updated
TC-32	Save profile with bad data (FR-22)	Patient is signed in	1) Open the profile page 2) Type an email in the wrong format 3) Hit Save	An error message shows and nothing gets saved	Worked as expected the validation message appeared
TC-33	Check a doctor's available times (FR-23)	Patient is signed in and the doctor has added availability	1) Open the doctors page 2) Pick a doctor 3) Look at their schedule	The doctor's free time slots are listed	Worked as expected the available times were shown
TC-34	Check availability for a doctor with no open slots (FR-23)	Patient is signed in	1) Open the doctors page 2) Pick a doctor who hasn't added	A "No available times" message appears	Worked as expected the "no available times" message showed

Test ID	Requirement (FR)	Preconditions	Test Steps	Expected Result	Actual Result
			any availability		
TC-35	Book an appointment (FR-24)	Patient is signed in and there's an open slot	1) Pick a doctor 2) Choose one of the open times 3) Confirm the booking	The appointment gets saved and shows up under My Appointments	Worked as expected the appointment was booked
TC-36	Try to book a slot someone else already took (FR-24)	The time is already taken	1) Try to book a time that's already been reserved	A conflict message appears and the booking is blocked	Worked as expected the booking was rejected with a message
TC-37	Reschedule an appointment (FR-25)	The appointment exists	1) Open My Appointments 2) Click the appointment 3) Pick a different open time 4) Save	The appointment shows the new time	Worked as expected the appointment was updated
TC-38	Reschedule to a time that's already booked (FR-25)	That time is already reserved	1) Open My Appointments 2) Click the appointment	A conflict message shows and the change isn't saved	Worked as expected the edit was rejected

Test ID	Requirement (FR)	Preconditions	Test Steps	Expected Result	Actual Result
			3) Pick a time that's taken 4) Save		
TC-39	Cancel an appointment (FR-26)	The appointment exists	1) Open My Appointments 2) Click the appointment 3) Hit Cancel 4) Confirm	The appointment is marked as cancelled	Worked as expected the appointment was cancelled
TC-40	Appointment notification (FR-27)	The patient has an appointment	1) Book or change an appointment 2) Open the notifications page	A notification with the appointment details appears	Worked as expected the notification showed up
TC-41	Start a chat with a doctor (FR-28)	Patient is signed in and has an appointment with the doctor	1) Pick the doctor 2) Click the chat button	A chat window with the doctor opens	Worked as expected the chat window opened
TC-42	Chat with a doctor the patient doesn't have an appointment with (FR-28)	No appointment exists between the patient and the doctor	1) Try to open a chat with the doctor	An access denied message appears	Worked as expected the chat was blocked

Test ID	Requirement (FR)	Preconditions	Test Steps	Expected Result	Actual Result
TC-43	Send a text message in chat (FR-29)	Chat with the doctor is open	1) Type something 2) Click Send	The message shows up in the chat and gets saved	Worked as expected the message was sent
TC-44	Send an empty message (FR-29)	Chat is open	1) Leave the text box empty 2) Click Send	Nothing gets sent and an alert pops up	Worked as expected the message wasn't sent
TC-45	Send an image or file (FR-30)	Chat is open	1) Click Attach 2) Pick a supported image or file 3) Send	The file shows up in the chat and gets saved	Worked as expected the file was sent
TC-46	Send an unsupported file type (FR-30)	Chat is open	1) Click Attach 2) Pick a file with an unsupported format 3) Send	An "unsupported file type" message appears	Worked as expected the file was rejected
TC-47	Submit a rating (FR-31)	The patient has had a past appointment	1) Open the rating page 2) rate 3) Submit	The rating or complaint is saved and a success message appears	Worked as expected the rating was saved
TC-48	Submit a rating with	The patient has had a past appointment	1) Open the rating page	A validation message appears and	Worked as expected the error message was shown

Test ID	Requirement (FR)	Preconditions	Test Steps	Expected Result	Actual Result
	missing info (FR-31)		2) Submit without a rating or any text	nothing is saved	
TC-49	Doctor gets a message notification (FR-32)	Doctor is signed in and the patient has an appointment with them	1) Have the patient send a message to the doctor 2) Open the doctor's notifications	A notification with the patient's name shows up	Worked as expected the doctor got the notification
TC-50	Doctor views patient messages (FR-33)	There are messages from patients to the doctor	1) Open the messages page 2) Pick a conversation 3) Read through the messages	The patient's messages are shown inside the conversation	Worked as expected the messages were displayed
TC-51	View messages when there aren't any (FR-33)	No conversations exist yet	1) Open the messages page	A "No messages" message is displayed	Worked as expected the empty state appeared
TC-52	Doctor replies to a patient's message (FR-34)	A message exists from a patient who has an appointment	1) Open the conversation 2) Type a reply 3) Send	The reply is saved and shown in the conversation	Worked as expected the reply was sent

3.9 Requirements Tracking Matrix:

It is a tool used in detailed analysis for a project to track and relate requirements to other elements of the system such as components, design, tests, and deliverables. The matrix aims to ensure that each requirement is properly met and tracked over the various stages of development. When creating a requirement tracking matrix, a list of requirements is listed in the first two rows or columns of the table. Other columns are then used to track information associated with each requirement, such as components that implement the requirement, related design, associated tests, and implementation status of the requirement.

ID	Title	Dependencies	Coding	Unit Test	Integration Test	System Test	Status	Notes
FR-37	Add Medicine Category (Pharmacist)	FR-64	Pharmacist/CategoryController@store Activity Sequence	TC-53, TC-54	TC-53	TC-53	Done	Pharmacist only
FR-38	Add Medicine (Pharmacist)	FR-37	Pharmacist/MedicineController@store Activity Sequence	TC-55, TC-56	TC-55	TC-55	Done	
FR-39	Edit / Delete Category	FR-37	Pharmacist/CategoryController@update, @destroy Activity Sequence	TC-57, TC-58	TC-57	TC-57	Done	
FR-40	Edit Medicine / View Inventory	FR-38	Pharmacist/MedicineController@index, @update Activity Sequence	TC-59, TC-60	TC-59	TC-59	Done	
FR-41	Search Inventory	FR-40	Pharmacist/MedicineController@index	TC-61, TC-62	TC-61	TC-61	Done	By name

ID	Title	Dependen cies	Coding	Unit Test	Integratio n Test	System Test	Status	Notes
			Activity Sequence					
FR-42	View Doctor Prescriptions	FR-36	Pharmacist/PrescriptionController@index Activity Sequence	TC-63	TC-63	TC-63	Done	
FR-43	Dispense Medicine	FR-42, FR-38	Pharmacist/DispenseController@store Activity Sequence	TC-64, TC-65	TC-64	TC-64	Done	
FR-44	Low Stock Alert	FR-38	Pharmacist/DashboardController@index Activity Sequence	TC-66	TC-66	TC-66	Done	At or below min quantity
FR-45	Track Medicine Expiry	FR-38	Pharmacist/MedicineController@update Activity Sequence	TC-67	TC-67	TC-67	Done	
FR-46	Daily Sales Report	FR-43	Pharmacist/ReportController@daily Activity Sequence	TC-68	TC-68	TC-68	Done	
FR-47	View Prescription Details	FR-42	Pharmacist/PrescriptionController@show Activity Sequence	TC-69	TC-69	TC-69	Done	
FR-48	Generate Invoice	FR-43	Pharmacist/DispenseController@invoice	TC-70	TC-70	TC-70	Done	

ID	Title	Dependencies	Coding	Unit Test	Integration Test	System Test	Status	Notes
			Activity Sequence					
FR-49	Record Damaged Returned Medicine	FR-38	Pharmacist/MedicineAdjustmentController@store Activity Sequence	TC-71, TC-72	TC-71	TC-71	Done	
FR-50	Classify Medicines by Category	FR-37, FR-38	Pharmacist/MedicineController@store, @update Activity Sequence	TC-73	TC-73	TC-73	Done	
FR-51	Monthly Sales Report	FR-43	Pharmacist/ReportController@monthly Activity Sequence	TC-74	TC-74	TC-74	Done	
FR-52	Chatbot Auto-Greeting	FR-19	Patient/ChatbotController@index Activity Sequence	TC-75	TC-75	TC-75	Done	
FR-53	Chatbot FAQs	FR-52	Patient/ChatbotController@reply Activity Sequence	TC-76	TC-76	TC-76	Done	
FR-54	Chatbot Working Hours	FR-52	Patient/ChatbotController@reply Activity Sequence	TC-77	TC-77	TC-77	Done	
FR-55	Chatbot Center Location	FR-52	Patient/ChatbotController@reply Activity Sequence	TC-78	TC-78	TC-78	Done	

ID	Title	Dependencies	Coding	Unit Test	Integration Test	System Test	Status	Notes
FR-56	Chatbot List Specialties	FR-52, FR-8	Patient/ChatbotController@reply Activity Sequence	TC-79	TC-79	TC-79	Done	
FR-57	Chatbot Booking Guidance	FR-52	Patient/ChatbotController@reply Activity Sequence	TC-80	TC-80	TC-80	Done	
FR-58	Chatbot List Doctors by Specialty	FR-56	Patient/ChatbotController@reply Activity Sequence	TC-81	TC-81	TC-81	Done	
FR-59	Chatbot Downloadable Files	FR-52	Patient/ChatbotController@reply Activity Sequence	TC-82	TC-82	TC-82	Done	
FR-60	Edit Doctor	FR-3	Admin/StaffController@update Activity Sequence	TC-83	TC-83	TC-83	Done	Admin only
FR-61	Edit Receptionist	FR-5	Admin/StaffController@update Activity Sequence	TC-84	TC-84	TC-84	Done	Admin only
FR-62	Add Pharmacist	FR-1	Admin/StaffController@store Activity Sequence	TC-85, TC-86	TC-85	TC-85	Done	Unique email
FR-63	Edit Pharmacist	FR-64	Admin/StaffController@update Activity Sequence	TC-87	TC-87	TC-87	Done	

ID	Title	Dependencies	Coding	Unit Test	Integration Test	System Test	Status	Notes
FR-64	Delete Pharmacist	FR-64	Admin/StaffController@destroy Activity Sequence	TC-88	TC-88	TC-88	Done	
FR-65	Allergy Alert (Doctor)	FR-35	Doctor/DoctorController@viewPatient Activity Sequence	TC-89	TC-89	TC-89	Done	Shown on patient view
FR-66	Profile Notification Popup (Patient)	FR-20	Patient/DashboardController@index Activity Sequence	TC-90	TC-90	TC-90	Done	Prompts to complete profile
FR-67	Separate Complaints and Ratings (Admin)	FR-29, FR-7	Admin/RatingController@index Activity Sequence	TC-91	TC-91	TC-91	Done	
FR-68	Track Patient Visit Count	FR-22	Patient::syncVisitsAndReward (Appointment hook) Activity Sequence	TC-92	TC-92	TC-92	Done	Counts completed visits
FR-69	Reward Every 4th Visit	FR-70	Patient::syncVisitsAndReward Activity Sequence	TC-93, TC-94	TC-93	TC-93	Done	
FR-70	Night Mode Toggle	FR-1	layouts/partials/night-mode.blade.php Activity Sequence	TC-95	TC-95	TC-95	Done	

Figure 33 chapter3 RTM

3.10 System Architecture:

System architecture refers to the overall structure and organization of components in a system. It aims to achieve objectives effectively and systematically, taking into account compatibility, scalability, security, maintainability, and performance. In a medical center management system, it includes components such as a database, user interfaces for different roles, appointment management, pharmacy management, and communication tools.

3.10.1 Client-Server Architecture:

The client-server pattern is an architectural pattern for software design, in which an application is divided into two main parts: the client and the server. The client interacts with the server by repeatedly sending requests and responses. In this project, the client is the web browser used by administrators, receptionists, doctors, pharmacists, and patients, while the server handles all data processing, business logic, and database operations using Laravel on the backend.

3.10.2 Model-View-Controller (MVC) Architecture:

In addition to the client-server pattern, this system is built using the MVC (Model-View-Controller) architectural pattern through the Laravel framework. MVC divides the application into three interconnected components:

1. **Model:** Manages the data and business logic. In this system, models represent entities such as Patient, Doctor, Appointment, Medicine, Prescription, and Invoice. Each model handles database interactions, validation rules, and relationships between tables.
2. **View:** Handles the presentation layer and user interface. Separate views are created for each user role — admin dashboard, receptionist appointment panel, doctor medical records page, pharmacist inventory interface, and patient portal.
3. **Controller:** Acts as an intermediary between Models and Views. Controllers process user requests, execute business logic through models, and return appropriate views. For example, the AppointmentController handles booking, rescheduling, and cancellation requests.

3.10.3 Features of this Architecture:

1. Separation of concerns: The system separates functions into different components frontend interfaces, backend business logic, database layer, and internal MVC structure. Medical data logic (Models), interface design (Views), and request handling (Controllers) are kept separate.
2. Ease of development: Laravel's MVC structure makes it clear to developers where each part of the code belongs, and the client-server model clarifies the roles of each component.
3. Ease of maintenance: With clear separation between components, fixing an issue in appointment booking does not affect the pharmacy module. Updates and repairs can be performed on each aspect separately.
4. Changeability: Changes can be made to one part of the system without affecting others. For example, updating the patient interface does not require changes to the doctor or admin interfaces.
5. Reusability: The same Model can be used by different Controllers and Views. The Patient Model is used by the receptionist for registration, the doctor for viewing history, and the patient for profile management.

3.10.4 Why Use this Architecture for this System?

1. Application performance optimization: Work is distributed between the client and server. The server handles database queries and business logic while the client handles interface rendering.
2. Increased security: User data and sensitive medical information are stored and processed on the server. Role-based access control is enforced server-side, making it difficult for unauthorized users to access protected data.

3. Scalability: New servers or resources can be added to handle increased load. As the medical center grows with more patients, doctors, and appointments, the system can scale accordingly.

4. Centralized data management: All patient records, medical histories, appointments, and pharmacy inventory are stored in a single centralized database, ensuring data consistency and eliminating duplication.

5. Multi-device accessibility The system can be accessed from any device with a browser — desktop, tablet, or smartphone — without installing additional software.

5. Testability: Each component can be tested independently — unit tests for Models, feature tests for Controllers, and interface tests for Views.

3.10.6 Client_Server Diagram /MVC Diagram :

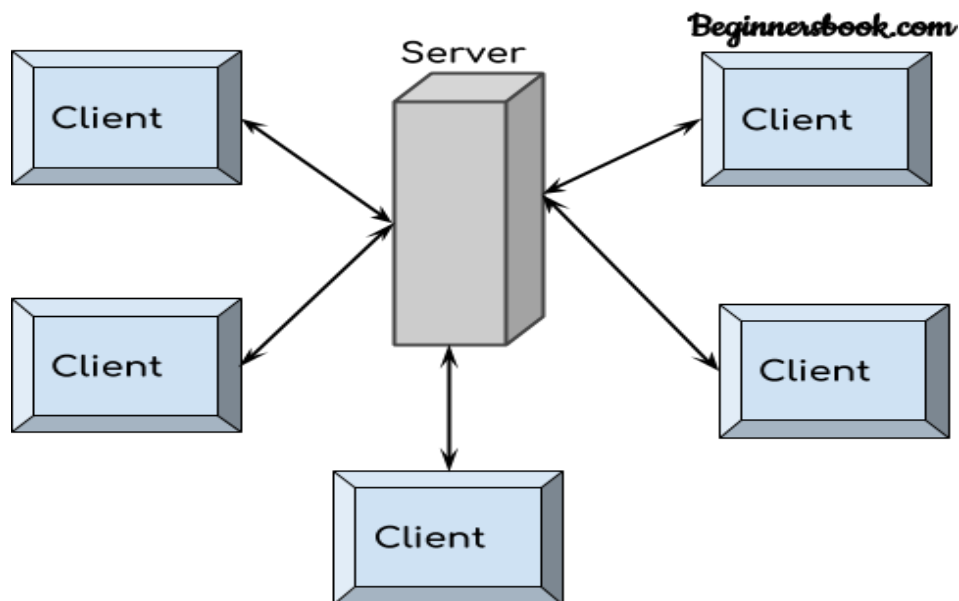


Figure 34 chapter3.34 client server

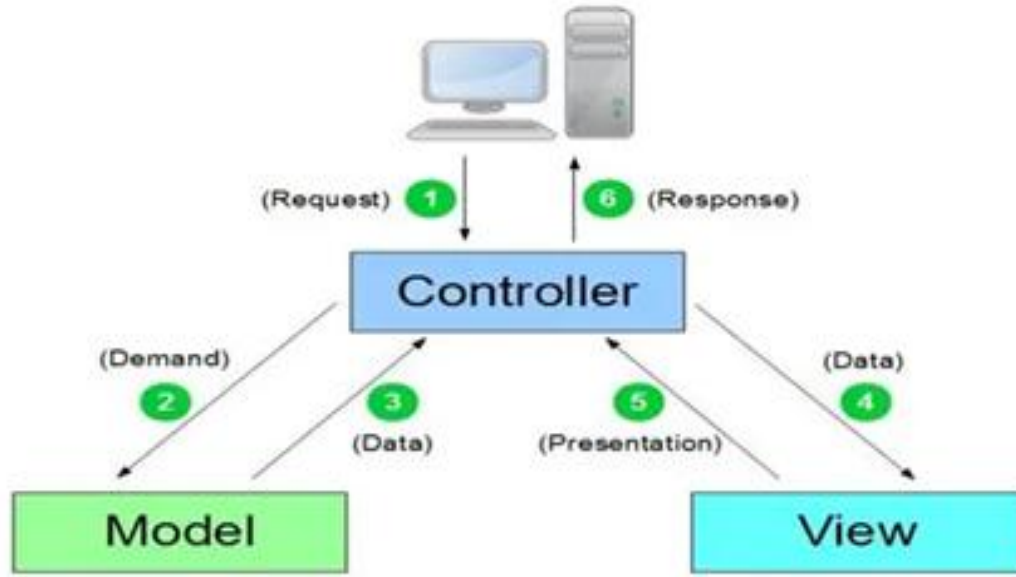


Figure 35 chapter3.35 mvc

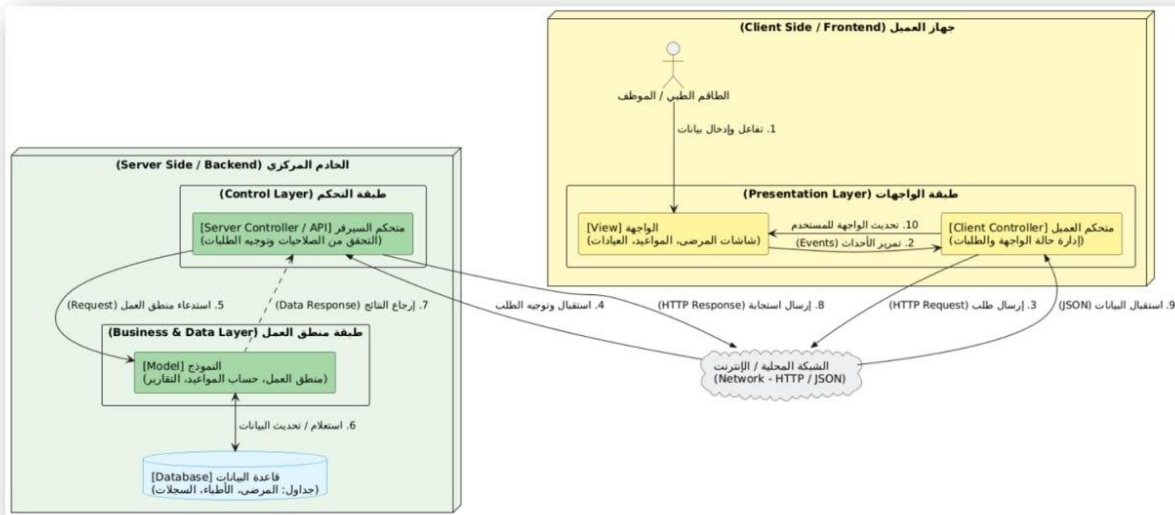


Figure 36 chapter3.36 mvc/client server

Chapter Four

The Design Phase of the Proposed System

4.1 Class Diagram(Low Level):

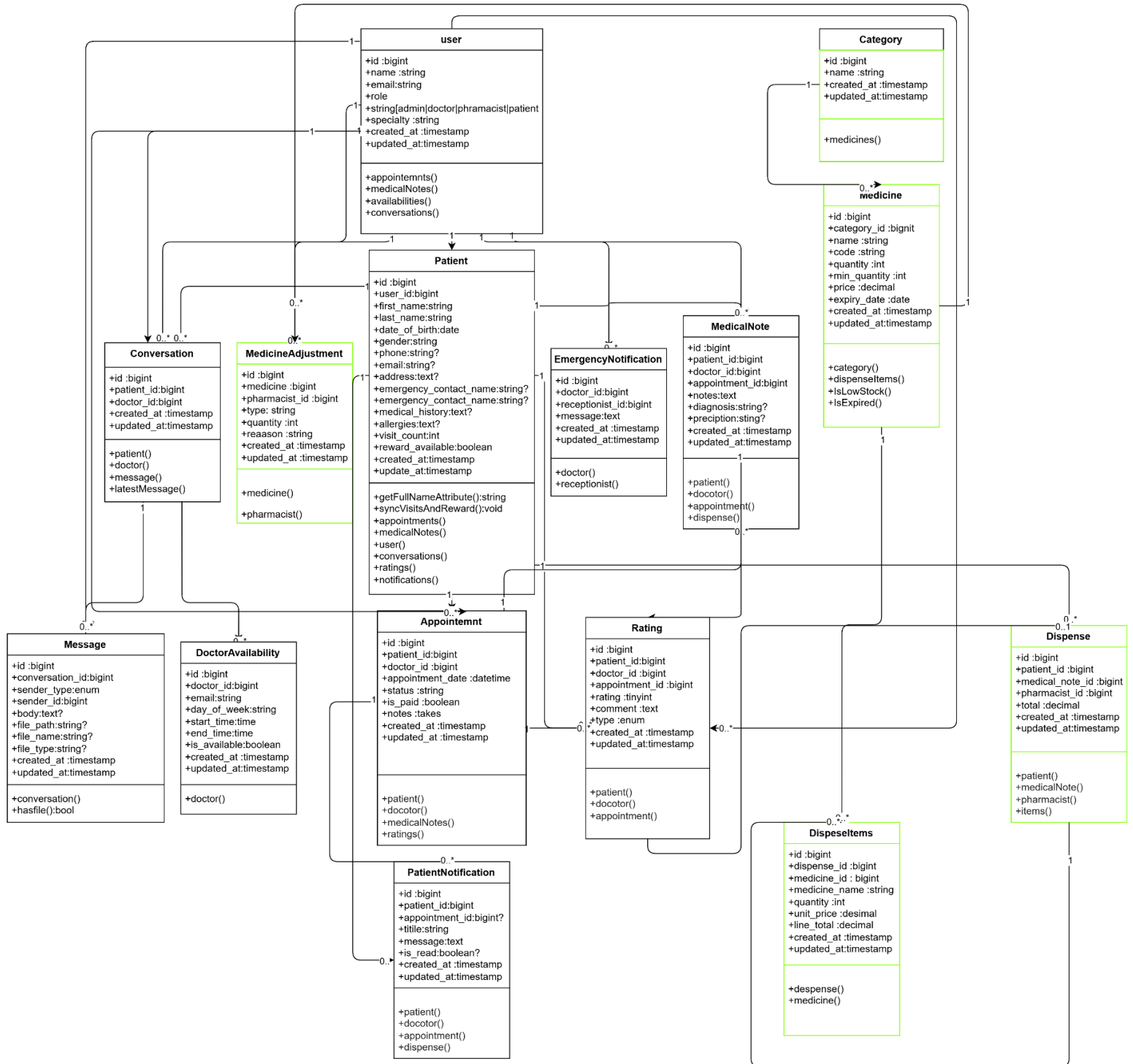


Figure 37 chapter4.1class diagram low level

4.2 Class Diagram(High Level):

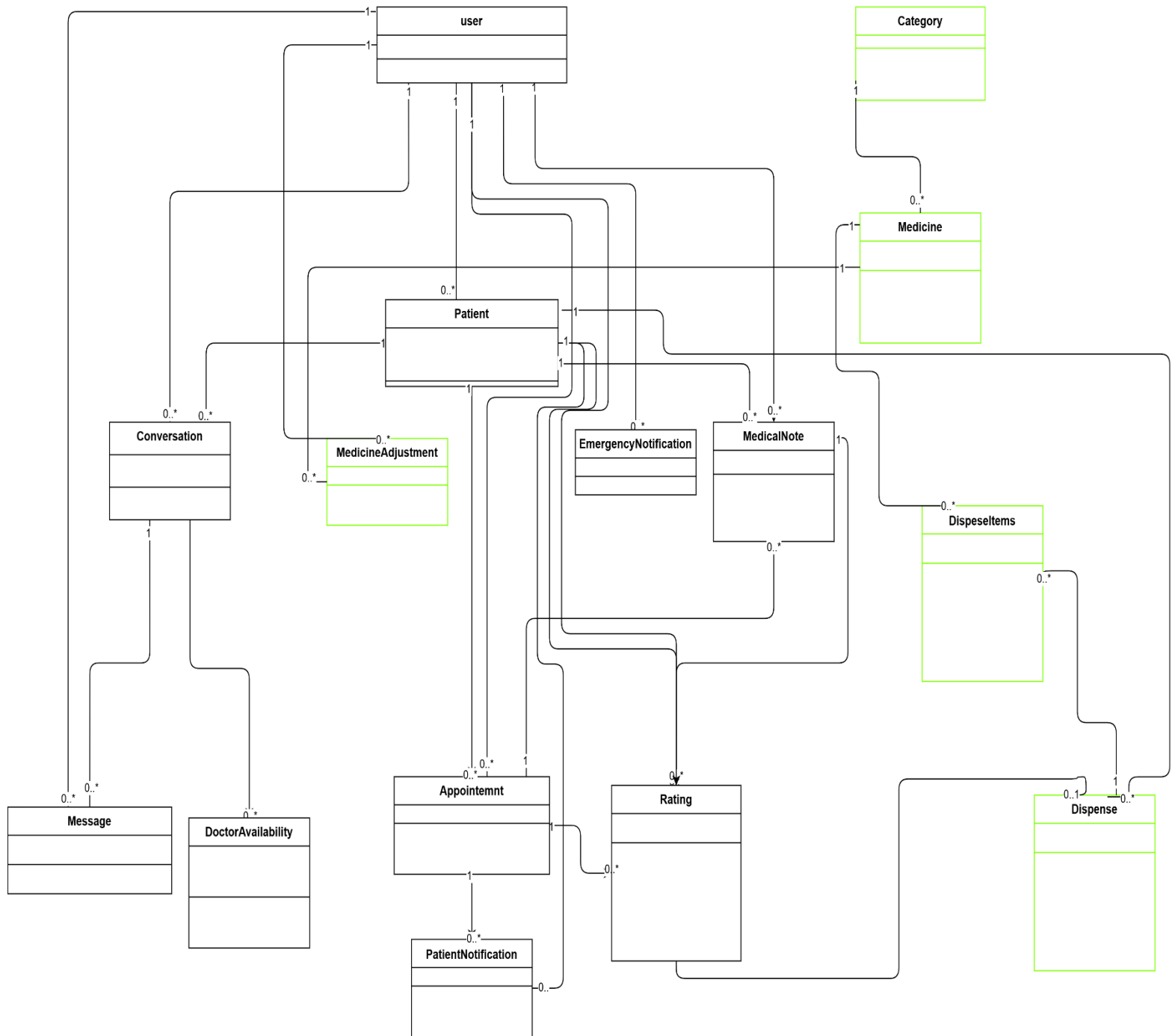


Figure 38 chapter4.2 class diagram high level

4.3 Design Models at the Physical Level:

1. The Devices Used:

1. Server: Used to host the application and databases. The server runs the Laravel backend, MySQL database, and serves the web application to all users.

2. Desktop Computers: Used by administrators, receptionists, doctors, and pharmacists within the medical center to access the system.

3. Laptop Computers: Used by doctors, staff and patients who may need to access the system from different locations.

2. Software and Technologies Used:

1. Programming Languages: PHP (backend logic), HTML, CSS, and JavaScript (frontend user interface), with Alpine.js for interactive components.

2. Databases: MySQL database management system to store all system data including patients, doctors, appointments, medical records, pharmacy inventory, prescriptions, invoices, messages, ratings, complaints, and notifications.

3. Web Server: such as Apache web server could be used to run the application and connect it to browsers.

4. Framework: Laravel framework (PHP) to facilitate application development, manage system activities, and implement the MVC architecture. Laravel provides built in features for authentication, routing, database migrations, and security.

5. Frontend Framework: Tailwind CSS for responsive and modern user interface design, ensuring compatibility across all devices.

6. Version Control: Git for source code management and collaboration.

3. Technical Infrastructure:

1. Internet: The project relies on an internet connection to allow users to access the platform. A local network can also be used within the medical center for internal staff access.

2. Security and Protection:

- Bcrypt encryption for password hashing .
- CSRF protection for all form submissions.
- SQL injection prevention through Laravel's Eloquent ORM and parameterized queries.
- Role-based access control to ensure each user type only accesses permitted features.
- Session management with expiration times.
- HTTPS protocol for encrypted data transmission.

3. Hosting: The application can be hosted on a local server within the medical center or on a cloud hosting service for remote access.

The above form contains technical details about the hardware, software, and infrastructure that will be used to implement this system at the physical level. These details can be adjusted depending on the medical center's specific needs and technical requirements.

4.4 System Interface Diagram

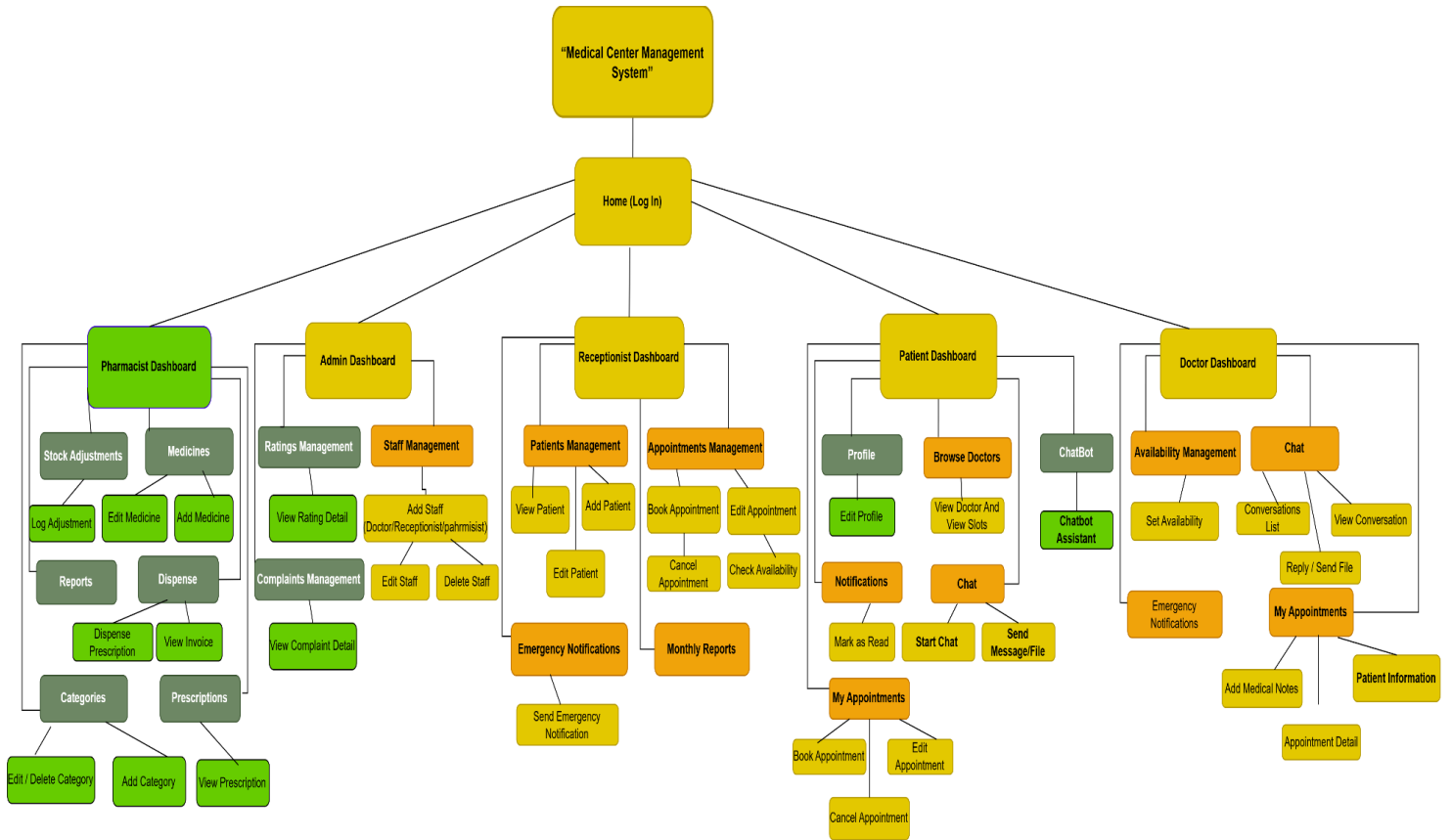


Figure 39 chapter4.3 System Interface Diagram

Chapter Five

Implementation of Project

5.1 Programming Resources:

5.1.1 Laravel Framework (Backend Framework):

Laravel is a powerful PHP framework designed for building modern web applications following the MVC architectural pattern. It provides expressive syntax, built-in security features, and a rich ecosystem for rapid development.

I have relied on this framework in our project because:

1. Great flexibility and ease of use.
2. Built-in authentication and role-based access control.
3. Eloquent ORM for easy database interactions.
4. Blade templating engine for clean and reusable views.
5. Built-in CSRF protection and SQL injection prevention.
6. Artisan CLI for automating common development tasks.
7. Extensive documentation and large community support.
8. Built-in queue system for handling notifications and emails.

5.1.2 Tailwind CSS (Frontend Framework):

Tailwind CSS is a utility-first CSS framework that allows building custom user interfaces rapidly. It provides low-level utility classes that give complete design control without overriding styles.

I have relied on this framework in our project because:

1. Highly customizable design without writing custom CSS.
2. Responsive design built-in for all screen sizes.
3. Compatible with all modern browsers.
4. Consistent and reusable design patterns.
5. Easy to learn and integrate with Laravel.
6. Small production file size after purging unused styles.

7. Works seamlessly with Alpine.js for interactive components.

5.1.3 Alpine.js (Frontend Interactivity):

Alpine.js is a lightweight JavaScript framework for adding interactive behavior to web pages. It is used alongside Tailwind CSS to handle dynamic UI components such as dropdown menus, modal windows, and form interactions without the complexity of larger frameworks.

5.2 Programming Tools:

5.2.1 Laravel Artisan (Command Line Tool):

The project was developed using Laravel's built-in command line tool called Artisan, which provides helpful commands for development.

Features:

1. Free and open source with a large community of users and developers.
2. Provides many built-in commands including:
 - `php artisan serve` to run the development server.
 - `php artisan make:model` to create models.
 - `php artisan make:controller` to create controllers.
 - `php artisan migrate` to run database migrations.
 - `php artisan queue:listen` to process queued jobs.
3. Multi-Platform: Supports (Windows, Linux, macOS).
4. Easy to use with simple and intuitive commands.

5.3 Technologies Used:

5.3.1 HTML5:

HTML, or Hypertext Markup Language, allows web users to create and structure sections, paragraphs, and links using elements, tags, and attributes. It is the foundation of all web pages in this system.



Figure 40
chapter4.4 html

5.3.2 CSS3:

Cascading Style Sheets (CSS) is a style sheet language used to control the presentation of web pages. In this project, Tailwind CSS handles most styling, but custom CSS3 is used where needed for specific design requirements. CSS controls colors, fonts, spacing, layout, and responsive design for different devices.

5.3.3 JavaScript:

JavaScript is a text-based programming language used both on the client-side and server-side that allows web pages to be interactive. In this project, Alpine.js is used to add interactive elements such as dropdown menus, modal windows, form validation, and dynamic content loading.

5.3.4 PHP (Hypertext Preprocessor):

PHP is the most widely used open source server-side scripting language used mainly in web development to create dynamic websites and applications. Laravel is built on PHP, and all backend logic in this system — including user authentication, appointment management, pharmacy operations, and chat functionality — is written in PHP. PHP handles database interactions, form processing, session management, and data encryption using bcrypt for secure password hashing.



Figure 41 chapter4.5
php

5.3.5 MySQL (Database):

MySQL is an open-source relational database management system used to store all system data. It handles tables for patients, doctors, appointments, medical records, prescriptions, medicines, invoices, chat messages, ratings, complaints, notifications, and loyalty rewards.



Figure 42
chapter4.6 MySQL

5.4 Interface Pages:

5.4.1 (All Users) Login Page:

On this page, users enter their username and password. The system verifies the credentials, and if correct, redirects the user to their role-specific dashboard. If the credentials are wrong, an error message is displayed.

5.4.2 (Admin) Dashboard:

On this page, the system displays the admin control panel showing all key information including total patients, doctors, appointments, pharmacy statistics, recent complaints, ratings, and data analysis.

5.4.3 (Admin) Manage Staff:

On this page, the admin can view, add, edit, and delete staff accounts including doctors, receptionists, and pharmacists.

5.4.3.1 (Admin) Add Staff:

On this page, the admin enters the required information for a new staff member within the existing fields and creates the account.

5.4.3.2 (Admin) Edit Staff:

On this page, the admin modifies the information of an existing staff member.

5.4.3.3 (Admin) Delete Staff:

On this page, the admin searches for a staff member and deletes them if they exist.

5.4.4 (Admin) View Ratings

On this page, the admin can review all ratings submitted by patients about doctors.

5.4.5 (Admin) View Complaints:

On this page, the admin can review all complaints submitted by patients, with options to mark them as reviewed or resolved.

5.4.6 (Admin) Profile:

On this page, the admin can view and modify their personal information including username, email, and password.

5.4.7 (Receptionist) Dashboard:

On this page, the receptionist can view today's appointments, register new patients, and access appointment management tools.

5.4.8 (Receptionist) Register Patient:

On this page, the receptionist enters the patient's personal information, medical history, and allergies to create a new patient record.

5.4.9 (Receptionist) Book Appointment:

On this page, the receptionist selects a patient, chooses a doctor, and picks an available time slot based on real-time doctor availability.

5.4.10 (Receptionist) Manage Appointments:

On this page, the receptionist can view, reschedule, or cancel appointments.

5.4.11 (Receptionist) Send Emergency Notification:

On this page, the receptionist can send real-time emergency alerts to doctors.

5.4.12 (Receptionist) View Monthly Reports:

On this page, the receptionist can view monthly appointment statistics and reports.

5.4.13 (Doctor) Dashboard:

On this page, the doctor views their daily schedule, upcoming appointments, and pending medical records to complete.

5.4.14 (Doctor) View Appointments:

On this page, the doctor can view all their scheduled and past appointments.

5.4.15 (Doctor) View Patient Medical History:

On this page, the doctor can view the complete medical history of a patient including past diagnoses, prescriptions, and visit notes.

5.4.16 (Doctor) Add Medical Record:

On this page, the doctor adds a diagnosis, medical notes, and prescribes medicines after completing an appointment.

5.4.17 (Doctor) Manage Availability:

On this page, the doctor sets their working days, working hours, and days off for the week.

5.4.18 (Doctor) Chat with Patients:

On this page, the doctor can view and respond to messages from patients, including viewing file attachments.

5.4.19 (Doctor) Profile:

On this page, the doctor can view and modify their personal information and account settings.

5.4.20 (Pharmacist) Dashboard:

On this page, the pharmacist views pharmacy statistics including low-stock alerts, expiring medicines, and today's sales summary.

5.4.21 (Pharmacist) Manage Categories:

On this page, the pharmacist can add, edit, and delete medicine categories.

5.4.22 (Pharmacist) Manage Medicines:

On this page, the pharmacist can add, edit, and update medicines including stock quantities, prices, and expiry dates.

5.4.23 (Pharmacist) View Prescriptions:

On this page, the pharmacist can view all prescriptions sent by doctors.

5.4.24 (Pharmacist) Dispense Prescription:

On this page, the pharmacist selects a prescription, views the prescribed medicines, dispenses them, and generates an invoice.

5.4.25 (Pharmacist) View Invoices:

On this page, the pharmacist can view generated invoices for dispensed medicines.

5.4.26 (Pharmacist) Manage Stock Adjustments:

On this page, the pharmacist logs damaged, expired, or returned medicines and adjusts stock quantities accordingly.

5.4.27 (Pharmacist) View Sales Reports:

On this page, the pharmacist can view daily and monthly sales reports with totals and statistics.

5.4.28 (Pharmacist) Profile:

On this page, the pharmacist can view and modify their personal information.

5.4.29 (Patient) Dashboard:

On this page, the patient views their upcoming appointments, medical history, loyalty rewards status, and recent notifications.

5.4.30 (Patient) Book Appointment:

On this page, the patient selects a doctor, views available time slots, and books an appointment.

5.4.31 (Patient) Manage Appointments:

On this page, the patient can view, reschedule, or cancel their upcoming appointments.

5.4.32 (Patient) View Medical History:

On this page, the patient can view all their past diagnoses, prescriptions, and visit records.

5.4.33 (Patient) Chat with Doctor:

On this page, the patient can send messages and file attachments to their doctor and view the conversation history.

5.4.34 (Patient) Rate Doctor:

On this page, the patient can rate their doctor with a score from 1 to 5 after an appointment.

5.4.35 (Patient) Submit Complaint:

On this page, the patient can submit a complaint about their experience.

5.4.36 (Patient) Chatbot Assistant

On this page, the patient can interact with an intelligent chatbot to get answers about working hours, doctor specialties, location, booking instructions, and FAQs.

5.4.37 (Patient) Notifications:

On this page, the patient can view all their notifications including appointment confirmations, reminders, and loyalty reward alerts.

5.4.38 (Patient) Profile:

On this page, the patient can view and modify their personal information, change password, and view loyalty rewards.

5.5 Interfaces Screen:

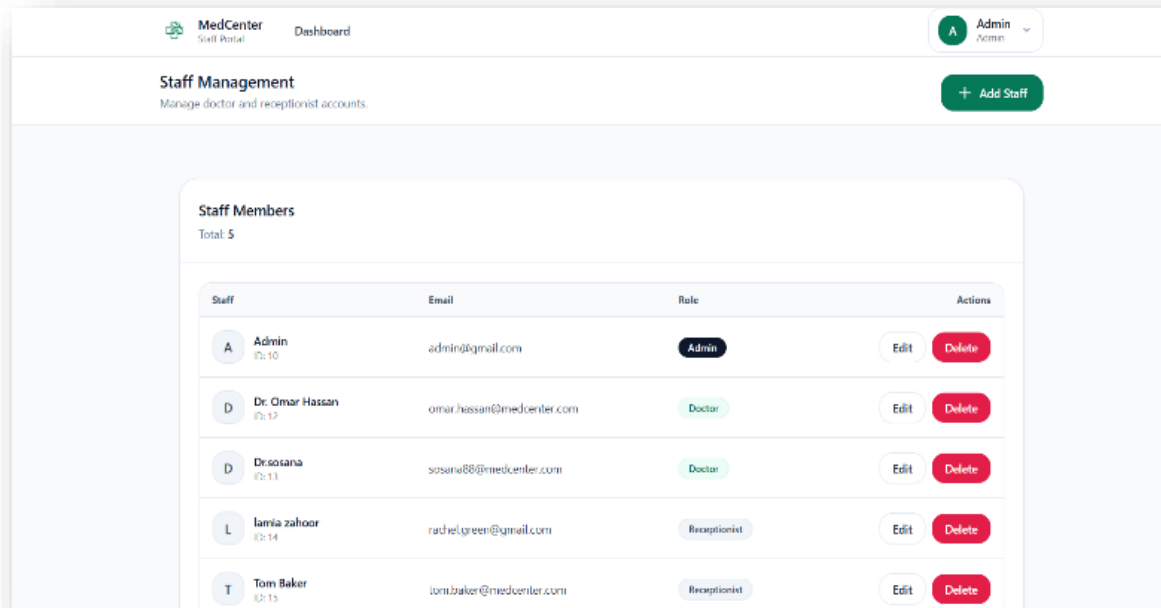


Figure 43 chapter4.7 Admin Dashboard

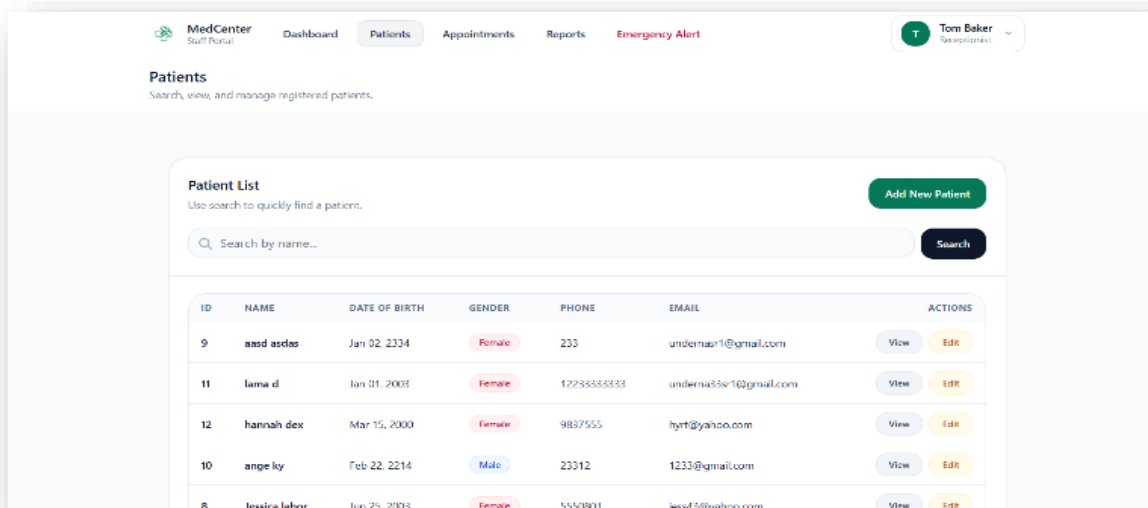


Figure 44 chapter4.8 Reception Dashboard

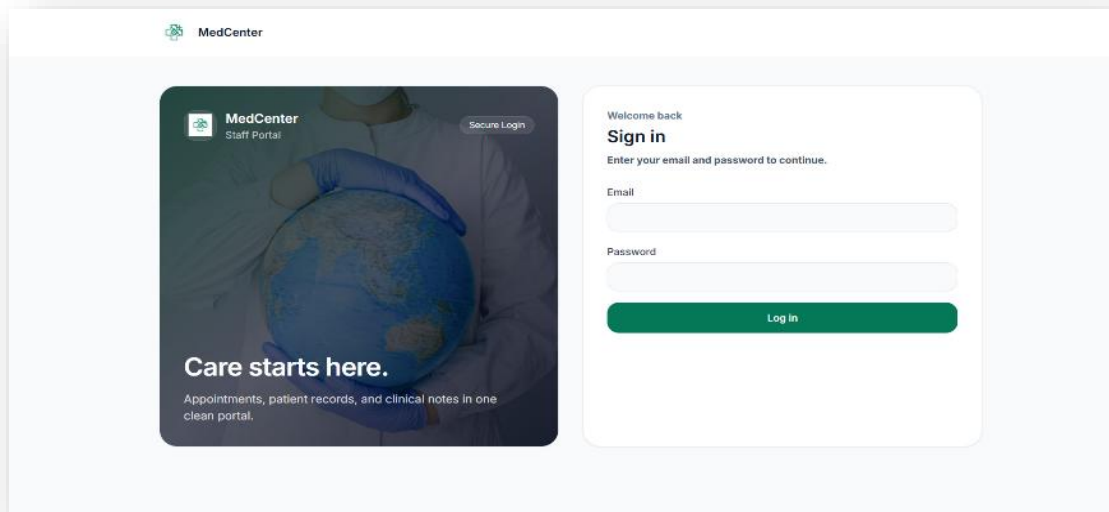


Figure 31 chapter4.9 Log in

The screenshot displays the 'Add New Patient' form in the MedCenter Staff Portal. The top navigation bar includes 'Dashboard', 'Patients', 'Appointments', 'Reports', and 'Emergency Alert'. The user 'Tom Baker' is logged in. The form is titled 'Register New Patient' and includes the instruction 'Register a patient. Optionally book an appointment right away.' The form fields are: First Name, Last Name, Date of Birth (with a calendar icon), Gender (a dropdown menu), Phone, Email, Address (with a text area icon), Emergency Contact Name, and Emergency Contact Phone.

Figure 32 chapter4.10 Add new patient

Chapter Six

Test and Result

6.1 Introduction:

The biggest challenge lies in verifying the correct functioning of the system and performing the required functions correctly, and this requires a sufficient amount of tests. The correct way to test any system must be carried out according to a proper methodology and successive stages to ensure the correctness of the workflow. Therefore, a set of tests has been implemented during the development stages of the system. When performing the test, it is ensured that there are no errors within the software because some of these errors remain latent until the system is running, which leads to the failure of the system to perform its functions. The errors detected vary in their degree of seriousness the error resulting from the software investigation stage is considered one of the easier mistakes to fix, while the error resulting from the lack of a correct understanding of the problem appears clearly in the analysis stage and leads to errors in all subsequent stages. The same is true for design — an error in this stage may be due to the designer's inexperience or not understanding the analysis properly.

6.1.1 Tests During Development:

6.1.1.1 Software Unit Testing:

PHPUnit is configured for unit testing in this project using Laravel's built-in testing framework. Unit tests are placed in the `tests/Unit` directory and test individual components in isolation. The project includes example unit tests that can be extended to cover all system modules such as patient registration, appointment booking, and pharmacy operations.

6.1.1.2 Integration Test:

Integration tests are placed in the `tests/Feature` directory and verify that different parts of the system work together correctly. The project includes tests for authentication, profile management, and other features to ensure that modules function properly when combined.

6.1.1.3 Quality Tests:

Quality tests were performed manually during development to verify that role-based access control works correctly — for example, ensuring that a patient cannot access admin pages, a

receptionist cannot access doctor functions, and each user type can only access their permitted features. Data validation, error messages, and overall system behavior were also tested.

6.1.2 Tests After Development:

6.1.2.1 Google Lighthouse:

Google Lighthouse was used to evaluate the system after development. The results were as follows: Performance scored 76, which indicates reasonable loading speed with room for optimization. Accessibility scored a perfect 100, confirming that the interfaces are fully accessible to all users. Best Practices scored 81, showing good adherence to modern web standards. SEO scored 92, demonstrating that the system is well-optimized for search engines. These results confirm that the system meets acceptable quality standards and provides a good user experience.

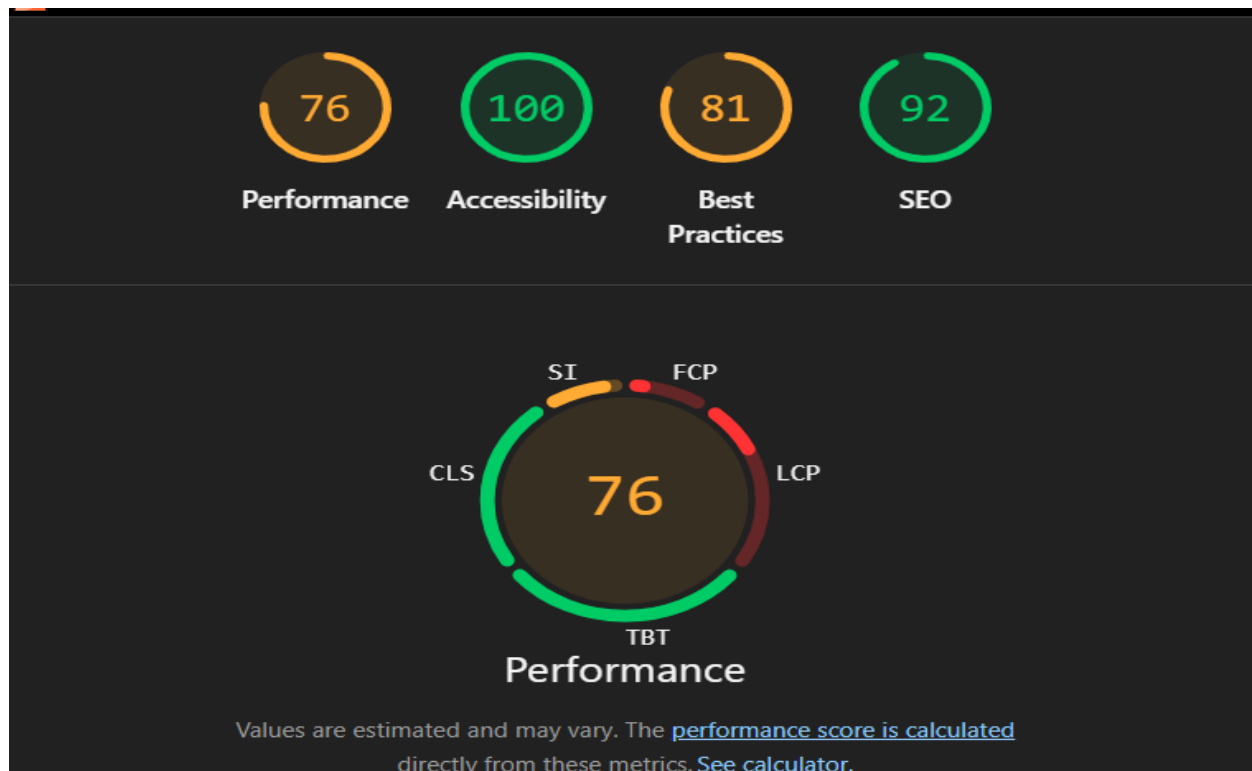


Figure 45 chapter6.1 Google test

Chapter Seven

Consultation and Future Prospects

7. Conclusion and Future Prospects

7.1 Conclusion:

Nowadays, traditional paper-based methods are becoming obsolete. We live in a new era where technology dominates every aspect of life. With modern software and hardware, manual errors are reduced and even eliminated. People prefer easy, fast, and secure solutions for every part of their lives, especially healthcare.

This project was designed to meet the requirements of a medical center management system. With this system, medical centers can provide convenient facilities for their staff and patients. Through this platform, we hope to reduce waste of time, avoid appointment conflicts and administrative errors, provide easy data flow between departments, improve patient satisfaction, and reduce the burden of manual paperwork.

We believe we have achieved our goals and are satisfied with the system we have developed. The system successfully integrates patient management, appointment scheduling, doctor workflows, pharmacy operations, patient-doctor communication, and administrative oversight into a single, efficient digital platform.

7.2 Future Prospects:

1. Develop mobile applications for Android devices to allow patients and doctors to access the system on the go.
2. Develop mobile applications for iOS devices to expand accessibility to all smartphone users.
3. Integrate an online payment gateway to allow patients to pay for appointments and pharmacy invoices electronically.
4. Expand the system to support multiple medical center branches with centralized data management.
5. Add video consultation support for remote doctor-patient appointments.
6. Implement AI-powered diagnostic suggestions to assist doctors in their decision-making process.
7. Integrate with government health databases and insurance providers for seamless data exchange.
8. Add multilingual support for patients and doctors who speak different languages.

Chapter Eight

Project Management

8.1 Project Description:

MedCenter is a web application built with the Laravel framework. The project aims to organize the daily operations of a clinic by managing patients, appointments, staff accounts, and appointment payment status. The system uses role-based access control to define what each user (Admin, Doctor, Receptionist or Patient) is allowed to do.

Git:

Version control for the project is managed using Git

GitHub file hosting

1. **Link:** <https://github.com/lemsdo-03/medcenter>
2. **Purpose:** Contains the full MedCenter project (backend + frontend).
3. **Structure:**
 - app/ (controllers, models, middleware)
 - routes/ (web routes)
 - database/ (data migrations and seeds)
 - resources/ (Blade views + assets)
 - Documentation files such as VERSION_CONTROL.md

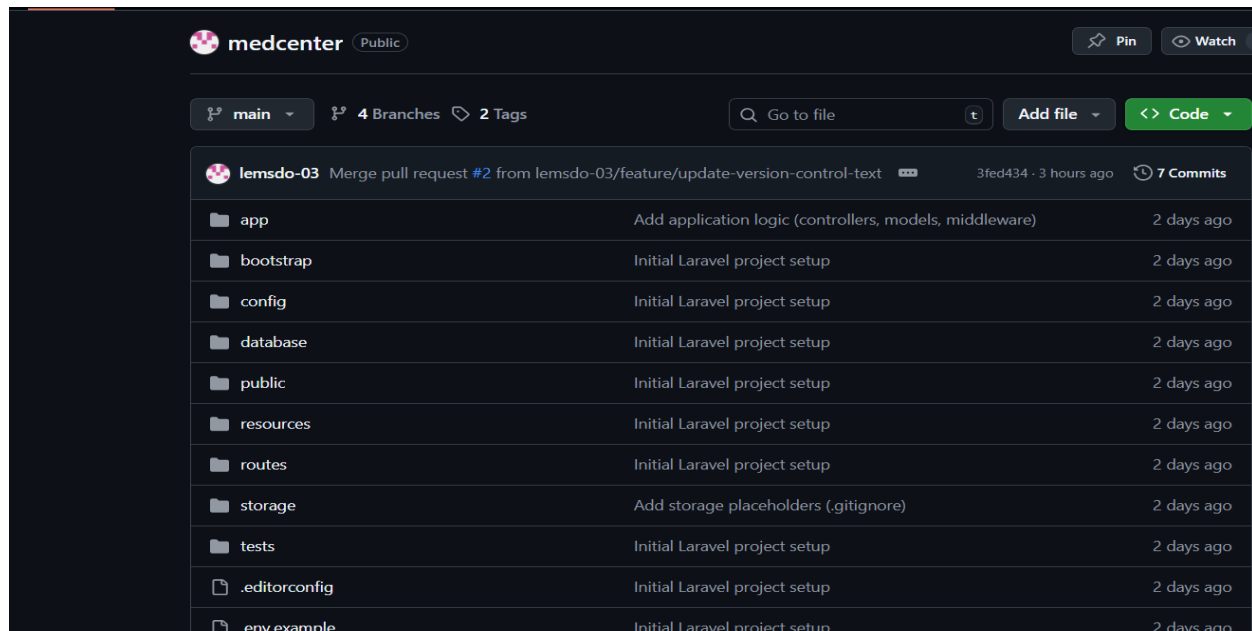


Figure 46 chapter8.1 github repository

8.2 Branch and Merge Strategy

Branches Used

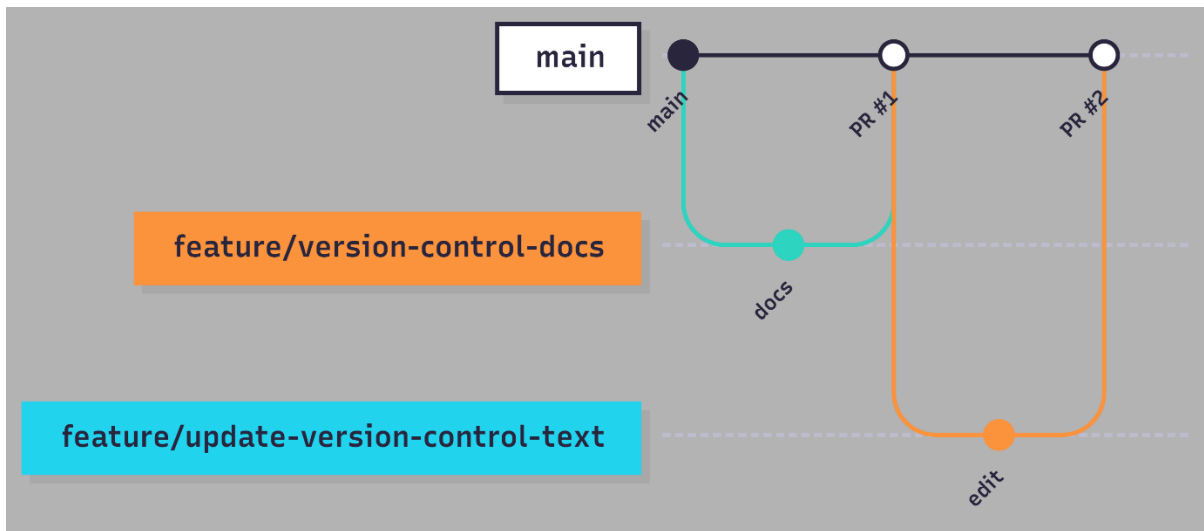
- main: the stable branch
- feature: temporary branches for features

Branch Workflow

1. A new feature branch is created from the main branch.
2. Changes are made and saved in the feature branch.
3. A pull request is created to merge the feature branch into the main branch.
4. After merging, the main branch contains the updated and stable version.

Examples of branches from this project

- feature/version-control-docs (merged into main)
- feature/update-version-control-text (merged into main).



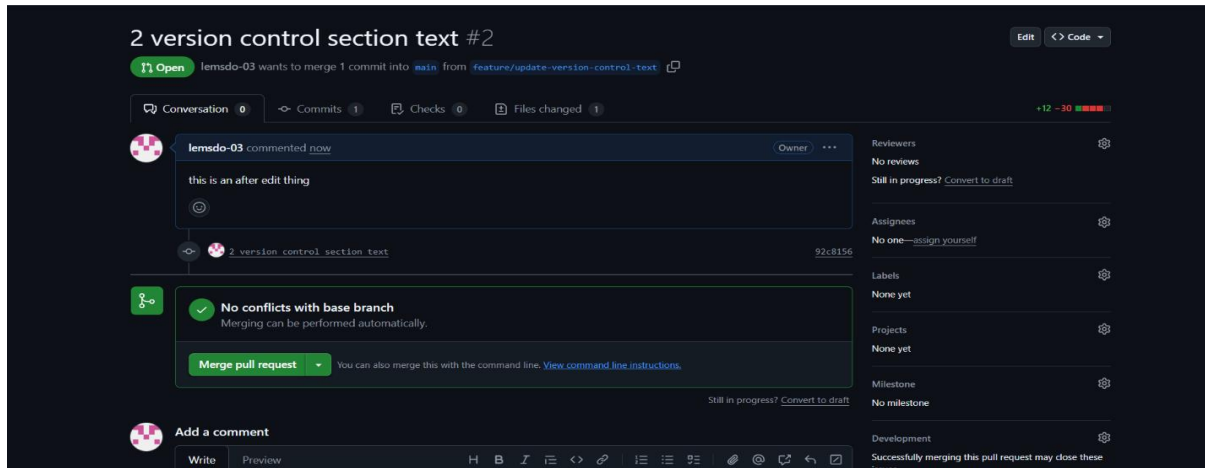
chapter8.2 branching47 Figure

8.3 Team Members and Responsibilities

The project was developed by a team consisting of two people. All team members participated in the stages of system analysis and architectural design, including requirements analysis, system modeling, and defining the overall system architecture.

Pull Request Guide:

Pull requests were used to merge feature branches into the main branch to keep changes organized and traceable..



8.4 Development Workflow

The development workflow of the MedCenter project follows a simplified task-based approach to keep changes organized and traceable using pull requests on GitHub.

The workflow can be summarized as follows:

1. Identify a development task: A new task or feature to be developed is identified.
2. Create a branch: A new branch named feature/* is created from the main branch.
3. Implementation and local testing: Changes are implemented and tested locally on the new branch.
4. Save changes: Commits are created with clear messages describing the updates made.
5. Push the branch: The feature branch is pushed to the GitHub repository.
6. Create a pull request: A pull request is created to merge the feature branch into the main branch.
7. Review and merge: After reviewing the changes and obtaining approval, the pull request is merged into the main branch.

8.5 Tags and Version Management

Git tags were used to identify important milestones of the project and stable delivery versions in the MedCenter repository.

Tagging helps to:

- Identify release points.
- Improve traceability.
- Enable the team to revert to a specific stable state of the code when needed.

The tags created:

- v1.0-final: The final baseline delivery version for the MedCenter project.
- v1.1-docs: An updated version for version control documentation and report enhancements.

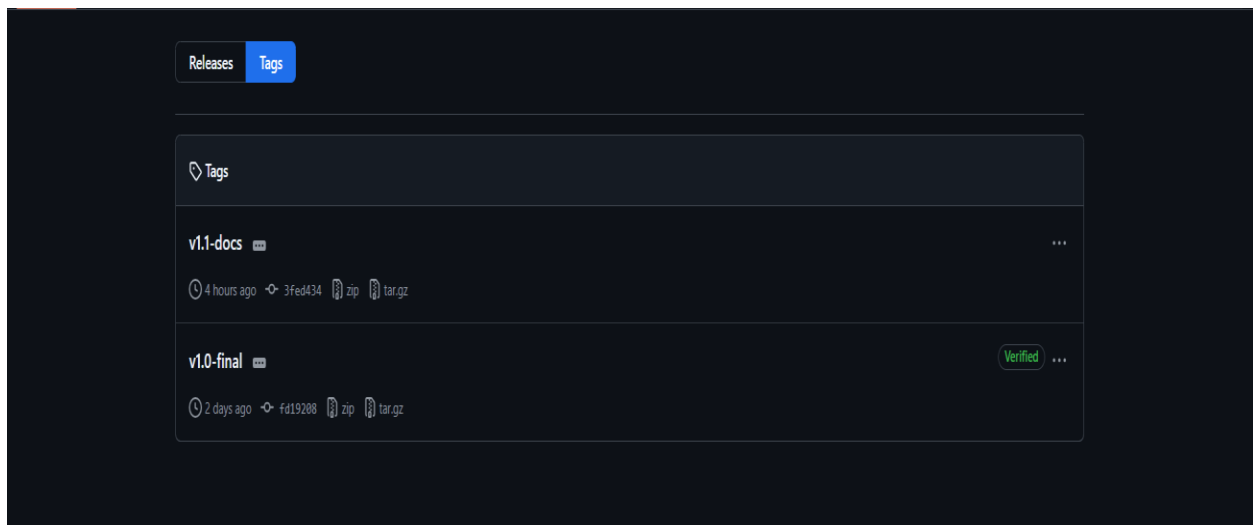


Figure 48 chapter8.3 tags

Chapter Nine

References

1. Laravel Documentation. Available at: <https://laravel.com/docs>
(Used for: Backend framework, MVC architecture, authentication, Eloquent ORM)
2. Tailwind CSS Documentation. Available at: <https://tailwindcss.com/docs>
(Used for: Frontend user interface design)
3. Alpine.js Documentation. Available at: <https://alpinejs.dev/docs>
(Used for: Interactive UI components)
4. MySQL Documentation. Available at: <https://dev.mysql.com/doc/refman/8.0/en/>
(Used for: Database design and management)
5. Vite Documentation. Available at: <https://vitejs.dev/docs/>
(Used for: Frontend build tool and asset bundling)
6. PHP Documentation. Available at: <https://www.php.net/docs.php>
(Used for: Server-side programming language)
7. PHPUnit Documentation. Available at: <https://phpunit.de/documentation.html>
(Used for: Software testing)
8. Google Lighthouse. Available at: <https://developer.chrome.com/docs/lighthouse/>
(Used for: Performance and quality testing)
9. draw.io / diagrams.net. Available at: <https://app.diagrams.net>
(Used for: Creating system diagrams)
10. Practo. Available at: <https://www.practo.com>
(Used for: Studying existing medical management platforms)
11. Zocdoc. Available at: <https://www.zocdoc.com>
(Used for: Studying appointment booking systems)
12. OpenEMR. Available at: <https://www.open-emr.org>
(Used for: Studying medical record management systems)